EF CORE PLUGIN DEVELOPMENT · REFERENCE AND PRACTICE

The EF Core Plugin Handbook

From the options extension to the NuGet package: everything you need for professional and even very complex extensions of Entity Framework Core. Explained throughout with a complete multi-tenancy plugin for PostgreSQL and SQL Server.

Edition	1.0
Versions	EF Core 10, Npgsql.EntityFrameworkCore.PostgreSQL 10, .NET 10
Sample	Complete solution at **github.com/guenni81/EF-Core-Plugin-Handbook**: core package, two provider packages, two sample apps, unit and integration tests
Verified	The sample code is built against EF Core 10 and Npgsql 10; unit and integration tests run against real PostgreSQL and SQL Server.
Structure	9 parts, 42 chapters, 4 appendices
License	MIT License, © 2026 guenni81. Free to use, copy, modify and share, including commercially; see the last page.

About this book

Many EF Core extensions start out as a handful of extension methods and then grow without a plan. If you understand how EF Core is built internally, however, you can write extensions that feel like part of EF Core itself: they are configured through the options, run inside the internal service provider, translate custom functions to SQL, generate migrations, write readable snapshots, and combine cleanly with other plugins.

This book explains each of these building blocks in two layers. First comes the **principle**: how does the mechanism work in EF Core, which variants exist, and what do you need to watch out for? Then comes the **implementation** in the sample plugin, with the code that is also in the repository. The chapters build on each other, but can also be read individually as a reference. The code in the repository and the snippets in this edition use English comments and messages. Each snippet's label names the file.

Box	Meaning
Important	Pitfalls that lead to incorrect behavior or security holes.
Tip	Proven practice that saves work or improves quality.
Note	Background, context and references to other chapters.

> **Public and internal APIs**
>
> Some APIs that plugins need live in namespaces ending in `.Internal`. EF Core calls them "pubternal": publicly visible, but without compatibility guarantees. The book marks these places, and chapter 24 explains how to deal with them.

Contents

I Foundations 5

1 What is an EF Core plugin? 6

2 EF Core’s architecture from a plugin’s point of view 7

3 The sample scenario 11

II Infrastructure 13

4 The options extension 14

5 Registering services 17

6 Public API and integration into the application 21

III The model 24

7 Annotations and Fluent API 25

8 Conventions 28

9 Model validation 33

10 Type mappings and value converters 34

IV Queries 37

11 The query pipeline 38

12 Translating methods and members 39

13 Preventing client-side evaluation 43

14 Further query extension points 44

V Run time 45

15 Interceptors 46

16 Diagnostics and logging 51

VI Migrations and design time 53

17 How migrations work 54

18 Declarative SQL from annotations 55

19 Custom migration operations 58

20 Design-time services 60

21 Compiled models, trimming and NativeAOT 62

22 Scaffolding (reverse engineering) 63

VII Quality and delivery 64

23 Test strategy 65

24 Versioning and compatibility 68

25 Packaging and documentation 70

26 The checklist from A to Z 71

VIII Advanced topics 74

27 Bulk operations and the limits of interceptors 75

28 Value generation 76

29 The update pipeline 78

30 Complex types and JSON 79

31 Models per runtime context 80

32 Advanced migrations 81

33 Row Level Security in production 82

34 Plugin performance 84

35 Using the plugin in ASP.NET Core 86

36 CI and API stability 89

IX Deep extensions 90

37 Custom SQL expression nodes 91

38 Aggregate translators 96

39 Property-level Fluent API and runtime annotations 98

40 Scaffolding in practice 101

41 What belongs in the core 103

42 A second provider: SQL Server 104

X Appendices 107

A Extension points and lifetimes 108

B Files of the sample plugin 109

C Glossary 111

D Debugging toolbox 112

License 114

Part I

Foundations

What a plugin is, how EF Core is built internally, and which scenario accompanies us through the book.

› Chapter 1: What is an EF Core plugin?

› Chapter 2: EF Core’s architecture from a plugin’s point of view

› Chapter 3: The sample scenario

1 What is an EF Core plugin?

The term “plugin” is used for very different things. For this book, it helps to distinguish four levels that differ in how deeply they reach into EF Core:

Level	Description	Examples
1 · Helpers	Extension methods on `DbContext` or `IQueryable` that only call public API.	Paging helpers, `WhereIf`, seeding methods
2 · Model helpers	Fluent API extensions and annotations, without services of their own inside EF Core.	Custom `HasXyz()` methods, ModelBuilder helpers
3 · Real plugins	Own services in the internal service provider: conventions, query translation, type mappings, migrations, design time.	NetTopologySuite plugin, NodaTime plugin, EFCore.NamingConventions
4 · Providers	A complete database integration including connection, update pipeline and SQL generation.	Npgsql, Pomelo (MySQL), SQLite, Cosmos

This book covers **level 3** completely. Levels 1 and 2 need no book of their own: they only use EF’s public API, and chapter 7 covers what is specific to Fluent API extensions. The tools of level 4 (such as replacing the SQL generator) are explained where plugins need them.

Who this book is for

For developers who already use EF Core confidently: `DbContext`, migrations, LINQ queries and dependency injection are assumed. No knowledge of EF’s internals is required; the book introduces them where they are needed. Experience with PostgreSQL helps with the sample, but every SQL construct is explained when it first appears. Chapter 2 ends with a complete mini plugin that shows all building blocks on a single page before the large sample begins.

What a level 3 plugin can do

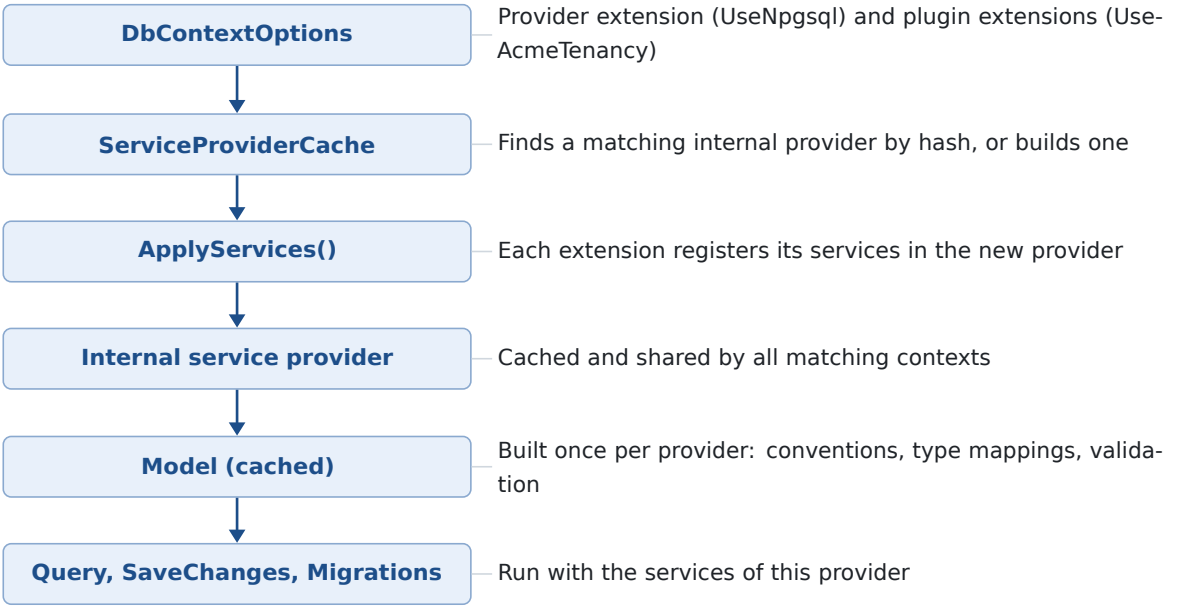
A complete plugin can extend EF Core at practically every point: it can make new CLR types available as column types, translate custom functions in LINQ to SQL, adapt the model automatically through conventions, accept metadata through the Fluent API and attributes, validate the model, intervene when saving and when opening connections, log its own events, add custom SQL to migrations and extend the `dotnet ef` tooling. The sample in this book uses every one of these possibilities.

> **Role models in the ecosystem**
>
> Before writing your own, look at existing plugins. **EFCore.NamingConventions** is a small, clean convention plugin. The **NetTopologySuite** plugin in the EF Core repository shows type mappings and translations. Npgsql’s **NodaTime** plugin shows a provider-specific plugin activated through the provider’s options builder.

2 EF Core’s architecture from a plugin’s point of view

Two containers: the internal service provider

Every `DbContext` resolves its infrastructure from an **internal** `IServiceProvider`. It is separate from the application’s DI container and contains several hundred services: type mapping, model building, the query pipeline, change tracking, the update pipeline, migrations. A plugin hooks in by registering additional services in this container or replacing existing ones.



Two caches are decisive here. The **service provider** is cached based on the options: contexts with the same configuration share one provider. The **model** is cached per provider (and context type). Both make EF Core fast, but they have direct consequences for plugins, which chapter 4 covers in detail.

Lifetimes

Every EF service has a fixed lifetime. **Singleton** here means: once per internal service provider, not per application. **Scoped** means: once per context instance. A singleton must not hold a scoped service, and a service that needs different data per context must not be a singleton. Appendix A lists the lifetimes of all extension points.

The dependencies pattern

EF services do not receive their dependencies individually, but as a parameter object, such as `MigrationsSqlGeneratorDependencies` or `ProviderConventionSetBuilderDependencies`. When you derive from an EF service, you accept this object in the constructor and pass it on. EF Core adds new dependencies to the object without breaking constructors. For plugins this is a blessing: derived services stay compatible across minor versions.

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (header)

```
public class TenancyMigrationsSqlGenerator : NpgsqlMigrationsSqlGenerator
{
    public TenancyMigrationsSqlGenerator(
        MigrationsSqlGeneratorDependencies dependencies,     // parameter object from EF
        INpgsqlSingletonOptions npgsqlSingletonOptions)      // additionally from the provider
        : base(dependencies, npgsqlSingletonOptions)
    {
    }
```

> **Code style in the sample**
>
> All classes in the sample use classic constructors with `private readonly` fields and `ArgumentNullException.ThrowIfNull`, no primary constructors. Their parameters are mutable throughout the class body, offer no natural place for validation and hide which state is stored. Classic constructors also match the style of EF Core and Npgsql, whose classes a plugin derives from. The only exception is positional records such as `RlsPolicy`, where the syntax generates real `init` properties.

Extend or replace?

There are two fundamentally different ways to extend EF Core. **Plugin interfaces** such as `IMethodCallTranslatorPlugin` or `IConventionSetPlugin` are designed for exactly this. They can be registered multiple times, EF queries all registrations, and several plugins coexist. **Replacing** means swapping a single service such as `IMigrationsSqlGenerator` for a derived class. That is sometimes unavoidable, but only one replacement can win per context.

> **Plugin interfaces before replacement**
>
> Always check first whether a plugin interface exists. Replacing a single service collides with every other plugin that replaces the same service and forces you to derive from the provider's implementation. That ties the plugin to one provider and its internal API. Chapter 5 shows how to stay composable anyway.

Warm-up: a complete plugin in five pieces

Before the large sample, here is the smallest plugin that is still complete. It makes PostgreSQL's `reverse()` available in LINQ as `EF.Functions.Reverse(...)`. Every later chapter elaborates one of its five pieces:

tests/Acme.EntityFrameworkCore.Tenancy.Tests/MinimalPluginTests.cs (1/2, shortened)

```
// 1. The function users call in LINQ; it is never executed
public static class ReverseDbFunctionsExtensions
{
    public static string Reverse(this DbFunctions _, string value)
        => throw new InvalidOperationException("Reverse can only be used in LINQ queries.");
}

// 2. The translator turns the method call into a SQL function
//    (chapter "Translating methods and members")
public class ReverseTranslator : IMethodCallTranslator
{
    // Constructor stores the ISqlExpressionFactory in _sql
```

```
    public SqlExpression? Translate(
        SqlExpression? instance, MethodInfo method, IReadOnlyList<SqlExpression> arguments,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
        => method == ReverseMethod
            ? _sql.Function("reverse", [arguments[1]], nullable: true,
                argumentsPropagateNullability: [true], typeof(string),
                arguments[1].TypeMapping)
            : null;
}

// 3. The plugin hands the translator to EF (chapter "Translating methods and members")
public class ReverseTranslatorPlugin : IMethodCallTranslatorPlugin
{
    public ReverseTranslatorPlugin(ISqlExpressionFactory sql)
    {
        Translators = [new ReverseTranslator(sql)];
    }

    public IEnumerable<IMethodCallTranslator> Translators { get; }
}
```

tests/Acme.EntityFrameworkCore.Tenancy.Tests/MinimalPluginTests.cs (2/2, shortened)

```
// 4. The options extension registers the plugin in EF's internal provider
//    (chapters "The options extension" and "Registering services")
public class ReverseOptionsExtension : IDbContextOptionsExtension
{
    private DbContextOptionsExtensionInfo? _info;

    public DbContextOptionsExtensionInfo Info => _info ??= new ExtensionInfo(this);

    public void ApplyServices(IServiceCollection services)
        => new EntityFrameworkRelationalServicesBuilder(services)
            .TryAdd<IMethodCallTranslatorPlugin, ReverseTranslatorPlugin>();

    public void Validate(IDbContextOptions options)
    {
    }

    // ExtensionInfo: not a provider, no settings -> hash 0, always the same provider
}

// 5. The Use method activates the plugin
//    (chapter "Public API and integration into the application")
public static DbContextOptionsBuilder UseReverse(this DbContextOptionsBuilder builder)
{
    ((IDbContextOptionsBuilderInfrastructure)builder).AddOrUpdateExtension(
        builder.Options.FindExtension<ReverseOptionsExtension>()
        ?? new ReverseOptionsExtension());
    return builder;
}
```

Usage

```
options.UseNpgsql(connectionString).UseReverse();

db.Customers.Where(c => EF.Functions.Reverse(c.Name) == "abc");
// SQL: WHERE reverse(c."Name") = 'abc'
```

That is all a working plugin needs. EF finds the translator because the extension registers it in the internal service provider, and every context configured with `UseReverse()` shares that provider. Because the argument references a column, the function needs no evaluatable expression filter (chapter 13 explains when it does). The test `MinimalPluginTests` checks the translation and that the same query fails without the plugin. The tenancy plugin consists of exactly these pieces, only many more of them.

3 The sample scenario

The sample plugin **Acme.EntityFrameworkCore.Tenancy** makes EF Core multi-tenant with PostgreSQL. The database itself enforces tenant separation through **Row Level Security** (RLS): every table gets a policy that only shows rows of the tenant whose ID is set in the current database session. Even a faulty query or raw SQL cannot read another tenant's data.

From the user's point of view it looks like this:

User's view

```
public class Order : ITenantOwned               // marker from the core package
{
    public int Id { get; set; }
    public TenantId TenantId { get; set; }      // strongly typed, automatically uuid
    public decimal Total { get; set; }
}

services.AddAcmeTenancy<HttpTenantProvider>();
services.AddDbContext<ShopContext>((sp, o) => o
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp, tenancy => tenancy.ApplyQueryFilter()));

// Querying and saving without any tenant logic in application code
db.Orders.Add(new Order { Total = 42 });         // the plugin sets TenantId
var mine = await db.Orders.ToListAsync();        // RLS filters in the database
```

To make this work, the plugin uses every extension point covered in this book:

Requirement	Implementation	Ch.
Configuration via `UseAcmeTenancy()`	Options extension, settings record, options builder	4–6
`TenantId` as a column type without configuration	Type mapping plugin, value converter, JSON reader/writer	10
`EF.Functions.CurrentTenant()` in LINQ	Method translator, evaluatable expression filter	12–13
`tenantId.IsEmpty` in LINQ	Member translator	12
RLS for all `ITenantOwned` entities	Convention with index and optional query filter	8
Fluent API, attribute `[NoTenantIsolation]`	Annotations, attribute convention, ConfigurationSource	7–8
Detect misconfiguration early	Validation convention, custom warnings	9, 16
Set TenantId automatically	SaveChanges interceptor	15
Tenant in the DB session	Connection interceptor	15
`ENABLE ROW LEVEL SECURITY`, `CREATE POLICY`	Annotation provider, migrations SQL generator	17–18
Roles and grants in migrations	Custom migration operations	19
Readable snapshots	Annotation code generator, design-time services	20
Compiled models and AOT	`Default` instances, source generators	21
`EF.Functions.IsNotDistinctFrom`, `EF.Functions.Every`	Custom SQL node, aggregate translator	37–38

Requirement	Implementation	Ch.
Tenant property with any name	`IsTenantProperty()`, runtime annotation	39
Scaffold RLS from an existing database	Custom database model factory	40
SQL Server	Second provider package on the same core	41–42

Package architecture

The plugin consists of three NuGet packages. The **core package** contains everything that is independent of the database and that assemblies with entity classes may reference without pulling in a provider. The **PostgreSQL package** is the complete implementation this book uses to explain everything. The **SQL Server package** uses the same core and shows in chapter 42 what a second provider has to implement itself.

Structure of the three packages

```
Acme.EntityFrameworkCore.Tenancy              Core (EF Core relational, no provider)
├─ TenantId, TenantIdConverter, TenantIdJsonReaderWriter, TenancyException
├─ ITenantOwned, ITenantProvider, NoTenantIsolationAttribute
├─ EF.Functions.CurrentTenant(), TenancyDbFunctionsAttribute
├─ Metadata/         runtime annotation, IsTenantProperty(), candidate lookup
├─ Conventions/      value generation for TenantId keys
├─ Query/            evaluatable expression filter (for all packages)
├─ Interceptors/     SaveChanges interceptor, abstract session interceptor
├─ Diagnostics/      event IDs, logging definitions, logger extensions
└─ DependencyInjection/  AddAcmeTenancy<T>()

Acme.EntityFrameworkCore.Tenancy.PostgreSQL    complete implementation
├─ Infrastructure/   options extension, builder, singleton options
├─ Storage/          TenantIdTypeMapping, type mapping plugin
├─ Query/            translators, custom SQL node, aggregate, SQL generator
├─ Metadata/         RLS annotations, RlsPolicy, annotation provider
├─ Builders/         Fluent API (HasRowLevelSecurity, HasPolicy, ...)
├─ Conventions/      isolation, attribute, validation, convention set plugin
├─ Migrations/       SQL generator, custom operations
├─ Design/           code generators, design-time services
├─ Scaffolding/      read RLS and policies from the database
└─ Runtime/          TenantConnectionInterceptor

Acme.EntityFrameworkCore.Tenancy.SqlServer     second provider
                                               (chapter "A second provider: SQL Server")
└─ Infrastructure/, Storage/, Query/, Metadata/, Conventions/, Migrations/, Runtime/
```

> **Tip**
>
> The folder structure follows EF Core's own (Infrastructure, Storage, Query, Metadata, Migrations, Design). Anyone who knows EF Core finds their way immediately, and when looking things up in the EF source you know where the counterpart lives.

Part II

Infrastructure

How a plugin is configured, registers its services and connects to the application's DI container.

› Chapter 4: The options extension

› Chapter 5: Registering services

› Chapter 6: Public API and integration into the application

4 The options extension

The entry point of every real plugin is a class implementing `IDbContextOptionsExtension`. It has three jobs: it carries the configuration, it registers the services in the internal provider in `ApplyServices`, and it describes itself to the cache through `Info`.

Configuration as a record

All model-relevant settings of the sample live in a record. Records provide `Equals` and `GetHashCode` over all fields, which is exactly what the cache key needs. New settings are added to the record without touching hashing or comparison.

Infrastructure/TenancySettings.cs

```
public sealed record TenancySettings
{
    public static TenancySettings Default { get; } = new();

    public string SettingName { get; init; } = "app.tenant_id";
    public string PolicyName { get; init; } = "tenant_isolation";
    public bool ForceRowLevelSecurity { get; init; } = true;
    public bool ApplyQueryFilter { get; init; }
    public bool CreateTenantIndex { get; init; } = true;
}
```

Immutability

EF compares and caches options. Once added, an extension must therefore never change again; every change creates a copy. The pattern from EF Core itself: a protected copy constructor and a virtual `Clone` method, so derived extensions clone their own type.

Infrastructure/TenancyOptionsExtension.cs (excerpt)

```
public class TenancyOptionsExtension : IDbContextOptionsExtension
{
    private DbContextOptionsExtensionInfo? _info;

    public TenancyOptionsExtension()
    {
    }

    protected TenancyOptionsExtension(TenancyOptionsExtension copyFrom)
    {
        ArgumentNullException.ThrowIfNull(copyFrom);

        Settings = copyFrom.Settings;
    }

    public virtual TenancySettings Settings { get; private set; } = TenancySettings.Default;

    public virtual DbContextOptionsExtensionInfo Info => _info ??= new ExtensionInfo(this);

    protected virtual TenancyOptionsExtension Clone() => new(this);
```

```
    public virtual TenancyOptionsExtension WithSettings(TenancySettings settings)
    {
        var clone = Clone();
        clone.Settings = settings;
        return clone;
    }
```

The info class and the service provider cache

The info class decides whether EF may reuse a cached internal service provider. Two methods must agree for this: `GetServiceProviderHashCode` as a fast pre-filter and `ShouldUseSameServiceProvider` as the exact comparison.

`Infrastructure/TenancyOptionsExtension.cs › ExtensionInfo`

```
private sealed class ExtensionInfo : DbContextOptionsExtensionInfo
{
    public ExtensionInfo(TenancyOptionsExtension extension)
        : base(extension)
    {
    }

    private new TenancyOptionsExtension Extension => (TenancyOptionsExtension)base.Extension;

    public override bool IsDatabaseProvider => false;

    public override string LogFragment
        => $"AcmeTenancy(Setting={Extension.Settings.SettingName}) ";

    public override int GetServiceProviderHashCode() => Extension.Settings.GetHashCode();

    public override bool ShouldUseSameServiceProvider(DbContextOptionsExtensionInfo other)
        => other is ExtensionInfo o && o.Extension.Settings == Extension.Settings;

    public override void PopulateDebugInfo(IDictionary<string, string> debugInfo)
    {
        debugInfo["Acme:Tenancy:Force"] =
            Extension.Settings.ForceRowLevelSecurity.ToString();
        // ... further settings
    }
}
```

There are two classic mistakes with these methods, with opposite consequences:

Mistake	Consequence
A model-relevant option is missing from the hash	Two contexts with different configuration share provider and model. The second gets the cached, **wrong** model. Often unnoticed in tests, because only one configuration runs there.
A value that varies per context is in the hash (tenant, connection string, instance)	A new internal provider including model is created for every context. Memory and startup time explode; EF warns after 20 providers with `ManyServiceProvidersCreatedWarning`.

> **The rule of thumb for the hash**
>
> The hash must contain exactly what **influences which services are registered or what the model looks like**, and nothing else. The tenant itself changes per request and must therefore never end up in the options; it arrives at run time through an interceptor (chapter 15).

`LogFragment` appears in EF's startup message ("initialized using provider … with options: …") and helps when troubleshooting. `PopulateDebugInfo` supplies values for EF's diagnostics when too many service providers are created. `IsDatabaseProvider` is `true` only for real providers.

Validation

`Validate` is called when the service provider is created. Here the plugin checks preconditions arising from the combination of extensions and reports them with a clear message instead of a later follow-up error:

Infrastructure/TenancyOptionsExtension.cs › Validate

```
public virtual void Validate(IDbContextOptions options)
{
    if (options.FindExtension<NpgsqlOptionsExtension>() is null)
        throw new InvalidOperationException(
            "UseAcmeTenancy() requires UseNpgsql(). Other databases need their own "
            + "provider package.");
}
```

5 Registering services

`ApplyServices` fills the internal service provider. EF provides a builder for this that knows the lifetime and cardinality of every EF service: `EntityFrameworkServicesBuilder` for core services and `EntityFrameworkRelationalServicesBuilder` additionally for relational services. It registers each service with the correct lifetime and automatically uses `TryAddEnumerable` for services that can be registered multiple times.

`Infrastructure/TenancyOptionsExtension.cs › ApplyServices`

```
public virtual void ApplyServices(IServiceCollection services)
{
    // 1) Plugin interfaces: multiple registrations, composable with other plugins
    new EntityFrameworkRelationalServicesBuilder(services)
        .TryAdd<IRelationalTypeMappingSourcePlugin, TenancyTypeMappingSourcePlugin>()
        .TryAdd<IMethodCallTranslatorPlugin, TenancyMethodCallTranslatorPlugin>()
        .TryAdd<IMemberTranslatorPlugin, TenancyMemberTranslatorPlugin>()
        .TryAdd<IEvaluatableExpressionFilterPlugin,
                TenancyEvaluatableExpressionFilterPlugin>()
        .TryAdd<IConventionSetPlugin, TenancyConventionSetPlugin>()
        .TryAdd<ISingletonOptions, ITenancySingletonOptions>(
            p => p.GetRequiredService<ITenancySingletonOptions>())
        // 2) Own services that EF does not know
        .TryAddProviderSpecificServices(m => m
            .TryAddSingleton<ITenancySingletonOptions, TenancySingletonOptions>()
            .TryAddSingleton<TenancyLoggingDefinitions, TenancyLoggingDefinitions>());

    // 3) Replace single services of the provider
    services.Replace(ServiceDescriptor
        .Scoped<IMigrationsSqlGenerator, TenancyMigrationsSqlGenerator>());
    services.Replace(ServiceDescriptor
        .Singleton<IRelationalAnnotationProvider, TenancyAnnotationProvider>());
}
```

Method	When	Why
`TryAdd<TService, TImpl>()`	EF services and plugin interfaces	Lifetime and multiple registration come from EF's table. Wrong lifetimes otherwise cause exceptions or subtly wrong behavior.
`TryAddProviderSpecificServices`	Own services EF does not know	The builder rejects unknown types; here you set the lifetime yourself.
`services.Replace(...)`	Replace a single provider service	The provider registers with `TryAdd`. `Replace` wins regardless of whether the plugin runs before or after the provider.

> **Why Replace works in both orders**
>
> EF calls the extensions' `ApplyServices` in the order they were added. If the plugin runs first, `Replace` adds the service and the provider's later `TryAdd` does nothing. If it runs afterwards, `Replace` replaces the provider's service. Both orders lead to the same result.

Singleton options

Singleton services such as type mapping plugins cannot resolve `IDbContextOptions` per context. For them there is `ISingletonOptions`: a singleton that EF initializes once from the options when creating the provider and validates for every further context.

Infrastructure/ITenancySingletonOptions.cs

```
public class TenancySingletonOptions : ITenancySingletonOptions
{
    public TenancySettings Settings { get; private set; } = TenancySettings.Default;

    // Once per internal service provider
    public virtual void Initialize(IDbContextOptions options)
        => Settings = options.FindExtension<TenancyOptionsExtension>()?.Settings
                      ?? TenancySettings.Default;

    // For every context: options must not differ from the provider's state
    public virtual void Validate(IDbContextOptions options)
    {
        var settings = options.FindExtension<TenancyOptionsExtension>()?.Settings
                       ?? TenancySettings.Default;
        if (settings != Settings)
            throw new InvalidOperationException("Tenancy options changed afterwards ...");
    }
}
```

The double registration in `ApplyServices` is required: once under its own interface so services can inject it, and once as `ISingletonOptions` so EF finds and initializes it. The factory variant ensures that both registrations return *the same* instance.

Staying composable despite replacement

When a plugin replaces a single service, a second plugin doing the same cannot be active at the same time. Professional plugins mitigate this by additionally offering the actual logic publicly and reusably. In the sample, `CreatePolicySql` is public and all generator methods are virtual. A user who must combine two plugins writes their own generator deriving from one and calling the logic of the other. This option belongs in the documentation.

Better still, a plugin avoids the conflict altogether by **decorating** the service instead of replacing it: it takes whatever implementation is registered at that point (the provider's or another plugin's), wraps it and delegates to it. An example is a (fictional) audit plugin that attaches a trigger to every new table. In its `ApplyServices` it looks up the existing registration and replaces only the descriptor, not the logic:

tests/Acme.EntityFrameworkCore.Tenancy.Tests/CompositionTests.cs › AuditOptionsExtension

```
public void ApplyServices(IServiceCollection services)
{
    // Registered so far: the provider's generator or that of another plugin
    var inner = services.LastOrDefault(d => d.ServiceType == typeof(IMigrationsSqlGenerator))
        ?? throw new InvalidOperationException("Call UseContosoAudit() after UseNpgsql().");

    services.Replace(ServiceDescriptor.Describe(
        typeof(IMigrationsSqlGenerator),
```

```
        sp => new AuditMigrationsSqlGenerator(
            (IMigrationsSqlGenerator)CreateInstance(sp, inner),
            sp.GetRequiredService<MigrationsSqlGeneratorDependencies>()),
        inner.Lifetime));     // lifetime of the original
}

// Creates the implementation the way the original descriptor would have
private static object CreateInstance(IServiceProvider sp, ServiceDescriptor descriptor)
    => descriptor.ImplementationInstance
       ?? descriptor.ImplementationFactory?.Invoke(sp)
       ?? ActivatorUtilities.CreateInstance(sp, descriptor.ImplementationType!);
```

The trick works because EF calls `ApplyServices` in the order in which the extensions were added; when the audit plugin runs, the provider has already registered its generator. `CreateInstance` reproduces the original registration exactly: an instance, a factory, or a type whose dependencies (such as Npgsql's singleton options) `ActivatorUtilities` resolves from the container. The decorator itself only uses the public interface:

tests/.../CompositionTests.cs › AuditMigrationsSqlGenerator (shortened)

```
public class AuditMigrationsSqlGenerator : IMigrationsSqlGenerator
{
    // Constructor: (IMigrationsSqlGenerator inner, MigrationsSqlGeneratorDependencies deps)

    public IReadOnlyList<MigrationCommand> Generate(
        IReadOnlyList<MigrationOperation> operations,
        IModel? model = null,
        MigrationsSqlGenerationOptions options = MigrationsSqlGenerationOptions.Default)
    {
        var commands = _inner.Generate(operations, model, options).ToList();   // unchanged
        var builder = new MigrationCommandListBuilder(_dependencies);
        var sql = _dependencies.SqlGenerationHelper;

        // Appended at the end: by then every table of this migration exists
        foreach (var table in operations.OfType<CreateTableOperation>())
        {
            builder.Append("CREATE TRIGGER audit_changes ")
                .Append("AFTER INSERT OR UPDATE OR DELETE ON ")
                .Append(sql.DelimitIdentifier(table.Name, table.Schema))
                .Append(" FOR EACH ROW EXECUTE FUNCTION audit.log_change()")
                .AppendLine(sql.StatementTerminator)
                .EndCommand();
        }

        commands.AddRange(builder.GetCommandList());
        return commands;
    }
}
```

In the application, the two plugins are combined without a single line of glue code:

Program.cs

```
services.AddDbContext<ShopContext>((sp, options) => options
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp)       // replaces IMigrationsSqlGenerator
    .UseContosoAudit());      // wraps whatever is registered: here Acme's
```

Decorate where possible, replace where necessary

Decorators compose with each other in any order, because each one wraps whatever it finds. A plugin that uses `Replace` however discards everything registered before it, including decorators. A replacing plugin therefore has to be activated first, and a decorator must check that there is something to decorate and fail with a clear message otherwise (the provider is missing or was added later).

Decorating requires that the public interface is enough. The tenancy plugin needs protected hooks inside `CREATE TABLE` and therefore derives from Npgsql's generator. For such cases the fallback described above remains: public logic such as `CreatePolicySql`, virtual methods, and `ReplaceService` in the application.

Tip

The test `CompositionTests` in the sample contains the complete audit plugin. It checks that the decorator wraps Acme's generator, that the migration contains the SQL of both plugins, and that activating the decorator before the provider fails.

6 Public API and integration into the application

The Use method

Users activate the plugin through an extension method on `DbContextOptionsBuilder`. It fetches the existing extension or creates a new one and registers it through `IDbContextOptionsBuilder` `Infrastructure`:

Infrastructure/TenancyDbContextOptionsBuilderExtensions.cs (excerpt)

```
public static DbContextOptionsBuilder UseAcmeTenancy(
    this DbContextOptionsBuilder optionsBuilder,
    Action<TenancyOptionsBuilder>? configure = null)
{
    ArgumentNullException.ThrowIfNull(optionsBuilder);

    var extension = optionsBuilder.Options.FindExtension<TenancyOptionsExtension>()
                    ?? new TenancyOptionsExtension();

    ((IDbContextOptionsBuilderInfrastructure)optionsBuilder).AddOrUpdateExtension(extension);
    configure?.Invoke(new TenancyOptionsBuilder(optionsBuilder));
    return optionsBuilder;
}

// Generic overload: method chaining stays typed
public static DbContextOptionsBuilder<TContext> UseAcmeTenancy<TContext>(
    this DbContextOptionsBuilder<TContext> optionsBuilder,
    Action<TenancyOptionsBuilder>? configure = null) where TContext : DbContext
    => (DbContextOptionsBuilder<TContext>)UseAcmeTenancy(
        (DbContextOptionsBuilder)optionsBuilder, configure);
```

The file lives in the namespace `Microsoft.EntityFrameworkCore`. That way the method can be found without an additional `using`, just like `UseNpgsql`. This is the common convention in the EF ecosystem.

The options builder

Settings are configured through a dedicated builder. It validates input immediately and writes every change back as a new extension:

Infrastructure/TenancyOptionsBuilder.cs (excerpt)

```
public class TenancyOptionsBuilder
{
    public TenancyOptionsBuilder(DbContextOptionsBuilder optionsBuilder)
    {
        ArgumentNullException.ThrowIfNull(optionsBuilder);

        OptionsBuilder = optionsBuilder;
    }

    protected virtual DbContextOptionsBuilder OptionsBuilder { get; }

    public virtual TenancyOptionsBuilder UseSettingName(string settingName)
```

```
        => With(s => s with
        {
            SettingName = TenancyIdentifiers.ValidateSettingName(settingName),
        });

    public virtual TenancyOptionsBuilder ApplyQueryFilter(bool apply = true)
        => With(s => s with { ApplyQueryFilter = apply });

    protected virtual TenancyOptionsBuilder With(
        Func<TenancySettings, TenancySettings> change)
    {
        var extension = OptionsBuilder.Options.FindExtension<TenancyOptionsExtension>()
                        ?? new TenancyOptionsExtension();

        ((IDbContextOptionsBuilderInfrastructure)OptionsBuilder)
            .AddOrUpdateExtension(extension.WithSettings(change(extension.Settings)));
        return this;
    }
}
```

Alternative: activation through the provider builder

Provider-specific plugins are sometimes activated through the provider's options builder instead, for example `UseNpgsql(cs, o => o.UseNodaTime())`. For this you write an extension method on `NpgsqlDbContextOptionsBuilder` and reach the actual builder through `IRelationalDbContextOptionsBuilderInfrastructure.OptionsBuilder`. The PostgreSQL package uses the simpler route via `DbContextOptionsBuilder`; the SQL Server package in chapter 42 shows the provider-builder style.

Connecting two containers

Some parts of a plugin need services of the *application*: the connection interceptor needs the current tenant, which only the application knows. The internal EF provider cannot resolve app services, though. The clean solution: such components live in the app container and are handed to EF via `AddInterceptors`.

DependencyInjection/… (core), Infrastructure/… (PostgreSQL), excerpts

```
// Core package: only register the tenant provider in the app container
public static IServiceCollection AddAcmeTenancy<TTenantProvider>(
    this IServiceCollection services,
    ServiceLifetime lifetime = ServiceLifetime.Scoped)
    where TTenantProvider : class, ITenantProvider
{
    services.TryAdd(ServiceDescriptor.Describe(
        typeof(ITenantProvider), typeof(TTenantProvider), lifetime));
    return services;
}

// Provider package: Use overload creates the interceptors from this provider
public static DbContextOptionsBuilder UseAcmeTenancy(
    this DbContextOptionsBuilder optionsBuilder,
    IServiceProvider applicationServices,
    Action<TenancyOptionsBuilder>? configure = null)
{
    var tenants = applicationServices.GetRequiredService<ITenantProvider>();
```

```
    return optionsBuilder
        .UseAcmeTenancy(configure)
        .AddInterceptors(
            new TenantConnectionInterceptor(tenants),
            new TenantSaveChangesInterceptor(tenants));
}
```

With `AddDbContext((sp, o) => ...)`, `sp` is the scope of the current request, because EF builds options per scope by default. The tenant provider and therefore the interceptors see the right tenant. The registration lives in the core package because it is the same for every provider (chapter 41). For the bridge in the other direction, an interceptor can read services from the *internal* provider via `eventData.Context.GetService<T>()`; the connection interceptor gets the setting name from the singleton options that way (chapter 15).

> **DbContext pooling**
>
> With `AddDbContextPool`, contexts are reused and receive their options only once. Scoped interceptors from the first scope would be reused for all later requests, with the tenant of the first request. For pooling, the tenant provider must be a singleton that determines the tenant anew on every access, for example via `IHttpContextAccessor`. Chapter 35 shows this variant, and the same applies to `AddDbContextFactory`, whose factory is a singleton by default.

Part III

The model

Metadata, Fluent API, conventions, validation and new column types.

- Chapter 7: Annotations and Fluent API
- Chapter 8: Conventions
- Chapter 9: Model validation
- Chapter 10: Type mappings and value converters

7 Annotations and Fluent API

Annotations are EF's mechanism for extensible metadata: key-value pairs on every model element (model, entity type, property, index, ...). A plugin stores its configuration as annotations, and the entire remaining infrastructure (conventions, migrations, snapshots, compiled models) can work with them.

Rules for annotations

Rule	Reason
Own prefix (`Acme:Rls:`)	Avoids collisions with EF, providers and other plugins.
Simple values only: string, bool, numbers, enums	Snapshots and migrations write values as C# literals. Custom classes cannot be serialized.
Complex values as a string (e.g. JSON)	The differ compares strings reliably via `Equals`. A record with a `string[]` would be compared by reference and constantly appear "changed".
Do not store default values	"Annotation missing" and "annotation has default" differ for the differ and produce empty migrations.
One annotation per piece of configuration	Every policy has its own annotation, so the diff shows exactly which one changed.

`Metadata/RlsAnnotationNames.cs, Metadata/RlsPolicy.cs (shortened)`

```
public static class RlsAnnotationNames
{
    public const string Prefix = "Acme:Rls:";
    public const string Enabled = Prefix + "Enabled";
    public const string Forced = Prefix + "Forced";
    public const string PolicyPrefix = Prefix + "Policy:";     // + policy name
    public const string QueryFilterKey = "Acme.Tenancy";
}

public sealed record RlsPolicy(string Name, PolicyCommand Command, bool Permissive,
    string[]? Roles, string? Using, string? WithCheck)
{
    public string Serialize()
        => JsonSerializer.Serialize(this, RlsJsonContext.Default.RlsPolicy);

    public static RlsPolicy Deserialize(string json) => /* ... */;
}
```

The Fluent API

Fluent API methods are extension methods on EF's builders. Three design rules make them professional: the **non-generic** variant is the actual implementation (the model snapshot uses non-generic builders, chapter 20). The **generic** variant only delegates and preserves the type for further chaining. And the class lives in the namespace `Microsoft.EntityFrameworkCore` so it is available everywhere without a `using`.

`Builders/RlsEntityTypeBuilderExtensions.cs (shortened)`

```
namespace Microsoft.EntityFrameworkCore;

public static class RlsEntityTypeBuilderExtensions
{
    public static EntityTypeBuilder HasRowLevelSecurity(
        this EntityTypeBuilder builder, bool force = false)
    {
        ArgumentNullException.ThrowIfNull(builder);
        builder.Metadata.SetAnnotation(RlsAnnotationNames.Enabled, true);

        if (force)
            builder.Metadata.SetAnnotation(RlsAnnotationNames.Forced, true);
        else
            builder.Metadata.RemoveAnnotation(RlsAnnotationNames.Forced);   // no default!

        return builder;
    }

    public static EntityTypeBuilder HasPolicy(
        this EntityTypeBuilder builder, string name, Action<PolicyBuilder> configure)
    {
        var policyBuilder = new PolicyBuilder(name);
        configure(policyBuilder);
        builder.Metadata.SetAnnotation(
            RlsAnnotationNames.PolicyPrefix + name, policyBuilder.Build().Serialize());
        return builder;
    }

    public static EntityTypeBuilder<TEntity> HasRowLevelSecurity<TEntity>(
        this EntityTypeBuilder<TEntity> builder, bool force = false) where TEntity : class
        => (EntityTypeBuilder<TEntity>)HasRowLevelSecurity(
            (EntityTypeBuilder)builder, force);

    // ... HasNoRowLevelSecurity, HasPolicy<TEntity> analogous
}
```

Nested configuration such as `HasPolicy` accepts an `Action<PolicyBuilder>`. The builder checks the database's rules already while the model is built. A mistake like "INSERT policy with USING" therefore surfaces on the application's first start, not when the migration runs in production:

Builders/PolicyBuilder.cs › Build

```
internal RlsPolicy Build()
{
    if (_using is null && _withCheck is null)
        throw new InvalidOperationException(
            $"Policy '{_name}' requires USING or WITH CHECK.");

    if (_command == PolicyCommand.Insert && _using is not null)
        throw new InvalidOperationException(
            $"INSERT policy '{_name}' only allows WITH CHECK.");

    if (_command is PolicyCommand.Select or PolicyCommand.Delete && _withCheck is not null)
        throw new InvalidOperationException(
            $"{_command} policy '{_name}' only allows USING.");

    return new RlsPolicy(_name, _command, _permissive,
```

```
        _roles.Count > 0 ? [.. _roles] : null, _using, _withCheck);
}
```

Configuration sources

EF remembers for every piece of metadata *where* it came from. A source can override an equally strong or weaker one, but never a stronger one. This is the mechanism through which plugins set sensible defaults that users can always override:

ConfigurationSource	Set by	Strength
`Explicit`	Fluent API in `OnModelCreating` (`Metadata.SetAnnotation`)	strongest
`DataAnnotation`	Attribute conventions (`fromDataAnnotation: true`)	medium
`Convention`	Conventions via `IConvention*Builder`	weakest

For custom conventions the plugin therefore also offers overloads on the convention builder. They return `null` when a stronger source decided otherwise:

Builders/RlsEntityTypeBuilderExtensions.cs › convention variant

```
public static IConventionEntityTypeBuilder? HasRowLevelSecurity(
    this IConventionEntityTypeBuilder builder,
    bool enabled,
    bool fromDataAnnotation = false)
    => builder.HasAnnotation(RlsAnnotationNames.Enabled, enabled, fromDataAnnotation)
           is null
        ? null
        : builder;
```

8 Conventions

Conventions change the model automatically. EF itself consists largely of conventions: discovering keys, finding relationships, deriving table names. A plugin adds its own to make configuration unnecessary.

Convention types and timing

A convention implements one or more interfaces. Each stands for an event during model building. The most important ones:

Interface	Event	Typical use
`IEntityTypeAddedConvention`	Entity type was added	Evaluate attributes, base configuration
`IPropertyAddedConvention`	Property was added	Column types, naming conventions
`IForeignKeyAddedConvention`	Relationship created	Indexes, delete behavior
`IEntityTypeBaseTypeChangedConvention`	Inheritance changed	Move configuration
`IModelFinalizingConvention`	Model is being finalized (still mutable)	Everything that depends on the finished model
`IModelFinalizedConvention`	Model is finished (read-only)	Validation

> **The most common convention mistake**
>
> Reactive conventions (`*Added`, `*Changed`) run while the user is still configuring. A column name may still change afterwards, an entity type may still get a base type. If you depend on that, you must react to *all* relevant events or use `IModelFinalizingConvention`. The latter is simpler and right for most plugins.

Plugging in conventions

The plugin interface `IConventionSetPlugin` receives the convention set and adds its own conventions. `ConventionSet.Add` registers a convention in all lists whose interfaces it implements, each time at the end:

`Conventions/TenancyConventionSetPlugin.cs`

```
public class TenancyConventionSetPlugin : IConventionSetPlugin
{
    private readonly ProviderConventionSetBuilderDependencies _dependencies;
    private readonly IDbContextOptions _options;
    private readonly ISqlGenerationHelper _sqlGenerationHelper;
    private readonly TenancyLoggingDefinitions _loggingDefinitions;

    public TenancyConventionSetPlugin(
        ProviderConventionSetBuilderDependencies dependencies,
        IDbContextOptions options,
        ISqlGenerationHelper sqlGenerationHelper,
        TenancyLoggingDefinitions loggingDefinitions)
    {
```

```
        // ... ArgumentNullException.ThrowIfNull for all parameters ...
        _dependencies = dependencies;
        _options = options;
        _sqlGenerationHelper = sqlGenerationHelper;
        _loggingDefinitions = loggingDefinitions;
    }

    public virtual ConventionSet ModifyConventions(ConventionSet conventionSet)
    {
        var settings = _options.FindExtension<TenancyOptionsExtension>()?.Settings;
        if (settings is null)
            return conventionSet;

        // chapter "Value generation"
        conventionSet.Add(new TenantIdKeyValueGenerationConvention());
        conventionSet.Add(new NoTenantIsolationAttributeConvention(_dependencies));
        conventionSet.Add(new TenantIsolationConvention(
            settings, _sqlGenerationHelper, _dependencies.Logger, _loggingDefinitions));
        conventionSet.Add(new TenancyModelValidationConvention(
            _dependencies.Logger, _loggingDefinitions));

        return conventionSet;
    }
}
```

`ProviderConventionSetBuilderDependencies` is the parameter object that EF's own conventions receive too. Among other things it contains the model logger, the validation logger and the type mapping source. Other services such as `ISqlGenerationHelper` can be injected as well, because the plugin itself is resolved from the internal provider.

The isolation convention

The sample's central convention runs at `ModelFinalizing` because column names are only final then. It finds the tenant property (marked with `IsTenantProperty()` or `TenantId` for `ITenantOwned`, chapter 39), enables RLS with a policy, creates an index, sets a runtime annotation for the SaveChanges interceptor and optionally a query filter:

Conventions/TenantIsolationConvention.cs › ProcessModelFinalizing

```
public virtual void ProcessModelFinalizing(
    IConventionModelBuilder modelBuilder,
    IConventionContext<IConventionModelBuilder> context)
{
    foreach (var entityType in modelBuilder.Metadata.GetEntityTypes())
    {
        if (entityType.BaseType is not null       // root types of a hierarchy only
            || entityType.IsOwned())              // owned types share the table
            continue;

        // Marked property or "TenantId" with ITenantOwned (core package,
        // chapter "Property-level Fluent API and runtime annotations")
        if (entityType.FindTenantPropertyCandidate() is not IConventionProperty property)
            continue;                             // (+ type check, shortened)

        if (entityType.IsRlsOptedOut())
        {
```

```
            _logger.TenantIsolationOptedOut(_loggingDefinitions, entityType);  // warning
            continue;
        }

        // No table (view, ToSqlQuery, keyless) -> no RLS
        if (StoreObjectIdentifier.Create(entityType, StoreObjectType.Table)
            is not { } table)
            continue;

        // Runtime annotation: this is how the SaveChanges interceptor finds the property
        entityType.Builder.HasAnnotation(
            TenancyAnnotationNames.TenantProperty, property.Name);

        ApplyRowLevelSecurity(entityType, property, table);
        if (_settings.CreateTenantIndex) EnsureTenantIndex(entityType, property);
        if (_settings.ApplyQueryFilter) ApplyQueryFilter(entityType, property);

        _logger.TenantIsolationApplied(_loggingDefinitions, entityType);       // debug
    }
}
```

Conventions/TenantIsolationConvention.cs › ApplyRowLevelSecurity

```
protected virtual void ApplyRowLevelSecurity(
    IConventionEntityType entityType,
    IConventionProperty property,
    StoreObjectIdentifier table)
{
    // Column name for exactly this table (may differ with table splitting)
    var column = _sqlGenerationHelper.DelimitIdentifier(property.GetColumnName(table)!);
    var condition =
        $"{column} = nullif(current_setting('{_settings.SettingName}', true), '')::uuid";

    var policy = new RlsPolicy(_settings.PolicyName, PolicyCommand.All, Permissive: true,
        Roles: null, Using: condition, WithCheck: condition);

    // Convention builder: does NOT override explicit configuration
    var builder = entityType.Builder;
    builder.HasAnnotation(RlsAnnotationNames.Enabled, true);
    if (_settings.ForceRowLevelSecurity)
        builder.HasAnnotation(RlsAnnotationNames.Forced, true);
    builder.HasAnnotation(RlsAnnotationNames.PolicyPrefix + _settings.PolicyName,
        policy.Serialize());
}
```

> **Validate SQL you generate yourself**
>
> The setting name is written into the policy expression without escaping. Unlike SQL that the developer writes in `Using(...)` themselves, here the *plugin* assembles SQL and is responsible for keeping invalid input out. That is why the options builder validates the name with a strict regular expression (`[a-z_][a-z0-9_]*\.[a-z_][a-z0-9_]*`).

Structural changes: index and query filter

Conventions may not only set annotations but change the model structurally. The index is only created if no index or key already starts with the tenant column. The query filter is built as an

expression tree and registered as a **named filter**, new in EF Core 10. Named filters coexist with the user's own filters and can be switched off selectively with `IgnoreQueryFilters(["Acme.Tenancy"])`.

Conventions/TenantIsolationConvention.cs (excerpt)

```
protected virtual void EnsureTenantIndex(
    IConventionEntityType entityType, IConventionProperty property)
{
    var covered = entityType.GetIndexes().Any(i => i.Properties[0] == property)
                || entityType.GetKeys().Any(k => k.Properties[0] == property);
    if (!covered)
        entityType.Builder.HasIndex(new[] { property });
}

protected virtual void ApplyQueryFilter(
    IConventionEntityType entityType, IConventionProperty property)
{
    // e => EF.Property<TenantId>(e, "...") == EF.Functions.CurrentTenant()
    // EF.Property works for any property name and for shadow properties too
    var parameter = Expression.Parameter(entityType.ClrType, "e");
    var body = Expression.Equal(
        Expression.Call(typeof(EF), nameof(EF.Property), [typeof(TenantId)],
            parameter, Expression.Constant(property.Name)),
        Expression.Call(CurrentTenantMethod, Expression.Property(null, FunctionsProperty)));

    entityType.Builder.HasQueryFilter(
        RlsAnnotationNames.QueryFilterKey, Expression.Lambda(body, parameter));
}
```

Notably, the filter uses the plugin's own function `EF.Functions.CurrentTenant()`. It is therefore translated to SQL by the translator from chapter 12. The plugin's building blocks interlock.

Final names, whoever sets them

The isolation convention reads the column name only at `ModelFinalizing` and only through `GetColumnName(table)`. Names that other conventions or plugins change are final by then. The sample checks this against EFCore.NamingConventions, which renames tables and columns to snake_case, in both activation orders:

Usage

```
options.UseNpgsql(connectionString).UseAcmeTenancy().UseSnakeCaseNamingConvention();
```

Generated migration SQL (excerpt, tests/.../NamingConventionTests.cs)

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders AS PERMISSIVE FOR ALL
    USING (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid)
    WITH CHECK (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid);
CREATE INDEX ix_orders_tenant_id ON orders (tenant_id);
```

> **Never assemble names yourself**
>
> A plugin that assembles names itself (from `Name` properties or by string concatenation) breaks as soon as a naming plugin is active. Always ask the model for the final name, and as late as possible.

Attribute conventions

Data annotations are evaluated with an attribute convention. EF provides the base class `TypeAttributeConventionBase<TAttribute>`, which finds the attribute and only asks for the reaction. The value is set with `fromDataAnnotation: true` so the Fluent API can override it:

`Conventions/NoTenantIsolationAttributeConvention.cs`

```
public class NoTenantIsolationAttributeConvention
    : TypeAttributeConventionBase<NoTenantIsolationAttribute>
{
    public NoTenantIsolationAttributeConvention(
        ProviderConventionSetBuilderDependencies dependencies)
        : base(dependencies)
    {
    }

    protected override void ProcessEntityTypeAdded(
        IConventionEntityTypeBuilder entityTypeBuilder,
        NoTenantIsolationAttribute attribute,
        IConventionContext<IConventionEntityTypeBuilder> context)
        => entityTypeBuilder.HasRowLevelSecurity(enabled: false, fromDataAnnotation: true);
}
```

> **Tip**
>
> The attribute itself lives in the *core package*, the convention in the provider package. Entity classes can use the attribute without depending on PostgreSQL. This is the same split as with EF's own attributes in `System.ComponentModel.DataAnnotations`.

9 Model validation

A plugin should detect misconfiguration before it leads to wrong SQL or data leaks. EF validates models through `IModelValidator`, but there is only one, and it belongs to the provider. For plugins, an `IModelFinalizedConvention` is the better place: it runs on the finished model and collides with nobody.

Conventions/TenancyModelValidationConvention.cs

```
public virtual IModel ProcessModelFinalized(IModel model)
{
    foreach (var entityType in model.GetEntityTypes())
    {
        if (!entityType.HasRlsConfiguration())
            continue;

        if (entityType.BaseType is not null)
            throw new InvalidOperationException(
                $"RLS on '{entityType.DisplayName()}' is invalid: only allowed on the root "
                + $"type ('{entityType.GetRootType().DisplayName()}').");

        if (StoreObjectIdentifier.Create(entityType, StoreObjectType.Table) is null)
            throw new InvalidOperationException("... not mapped to a table ...");

        if (!entityType.IsRlsEnabled() && entityType.GetRlsPolicies().Count > 0)
            _logger.PoliciesWithoutRowLevelSecurity(_loggingDefinitions, entityType);
    }
    return model;
}
```

The validation distinguishes two kinds of problems. **Structural errors** that are guaranteed to lead to wrong behavior throw an exception. **Questionable configuration** that may be intended produces a warning through the logging system. Users can turn such warnings into errors selectively (chapter 16).

10 Type mappings and value converters

A type mapping connects a CLR type with a database type. It determines the column type in migrations, how parameters are set, what literals look like in SQL, how values are compared and copied and how they are stored in JSON columns. For custom types a plugin has two options:

Approach	Advantages	Disadvantages
Value converter (user configures `HasConversion`)	No plugin needed	Must be configured for every property or via `ConfigureConventions`
`IRelationalTypeMapping` `SourcePlugin`	Type works everywhere without configuration, own literals, full control	Provider-specific (store type)

The type and its converter

The strongly typed `TenantId` and its converter live in the core package. The converter has a shared instance; the section on compiled models explains why.

TenantId.cs, TenantIdConverter.cs (shortened)

```
public readonly record struct TenantId(Guid Value)
{
    public static TenantId Empty => default;
    public bool IsEmpty => Value == Guid.Empty;              // translated to SQL
    public static TenantId New() => new(Guid.CreateVersion7());
}

public sealed class TenantIdConverter : ValueConverter<TenantId, Guid>
{
    public static TenantIdConverter Instance { get; } = new();

    public TenantIdConverter()
        : base(id => id.Value, value => new TenantId(value))
    {
    }
}
```

The type mapping

```
public class TenantIdTypeMapping : RelationalTypeMapping
{
    // Required for compiled models: public static Default property
    public static TenantIdTypeMapping Default { get; } = new();

    public TenantIdTypeMapping()
        : base(new RelationalTypeMappingParameters(
            new CoreTypeMappingParameters(
                typeof(TenantId),
                converter: TenantIdConverter.Instance,
                jsonValueReaderWriter: TenantIdJsonReaderWriter.Instance),
            storeType: "uuid",
```

```
            dbType: System.Data.DbType.Guid))
    {
    }

    protected TenantIdTypeMapping(RelationalTypeMappingParameters parameters)
        : base(parameters)
    {
    }

    // EF clones mappings, e.g. for facets or changed comparers
    protected override RelationalTypeMapping Clone(
        RelationalTypeMappingParameters parameters)
        => new TenantIdTypeMapping(parameters);

    protected override string GenerateNonNullSqlLiteral(object value)
    {
        var guid = value switch
        {
            TenantId id => id.Value,
            Guid g => g,
            _ => throw new ArgumentException($"Unexpected type {value.GetType()}."),
        };
        return $"'{guid}'::uuid";
    }
}
```

Every custom mapping needs the protected constructor taking `RelationalTypeMappingParameters` and a `Clone` override that creates its own type. Without it, a cloned mapping loses its behavior, for example literal generation.

The type mapping plugin

`RelationalTypeMappingSource` asks all plugins *before* falling back to the provider's mappings. A plugin returns `null` when it is not responsible:

Storage/TenancyTypeMappingSourcePlugin.cs

```
public class TenancyTypeMappingSourcePlugin : IRelationalTypeMappingSourcePlugin
{
    public virtual RelationalTypeMapping? FindMapping(
        in RelationalTypeMappingInfo mappingInfo)
    {
        if (mappingInfo.ClrType != typeof(TenantId))
            return null;

        // Explicitly different column type (HasColumnType("text")): not responsible
        var storeTypeBase = mappingInfo.StoreTypeNameBase;
        if (storeTypeBase is not null
            && !string.Equals(storeTypeBase, "uuid", StringComparison.OrdinalIgnoreCase))
            return null;

        return TenantIdTypeMapping.Default;
    }
}
```

> **Note**
>
> Without a mapping, EF would not recognize `TenantId` as a scalar type during model building and would abort with an error. With the plugin, the type works as a property, as a key, in queries, as a parameter and in JSON columns, without a single line of configuration.

JSON support

For JSON columns a mapping needs a `JsonValueReaderWriter`. It has three duties: read, write, and describe how it is created in generated code. The pattern follows EF's own `JsonGuidReaderWriter`:

TenantIdJsonReaderWriter.cs

```
public sealed class TenantIdJsonReaderWriter : JsonValueReaderWriter<TenantId>
{
    private static readonly PropertyInfo InstanceProperty =
        typeof(TenantIdJsonReaderWriter).GetProperty(nameof(Instance))!;

    public static TenantIdJsonReaderWriter Instance { get; } = new();

    private TenantIdJsonReaderWriter()
    {
    }

    public override TenantId FromJsonTyped(
        ref Utf8JsonReaderManager manager, object? existingObject = null)
        => new(manager.CurrentReader.GetGuid());

    public override void ToJsonTyped(Utf8JsonWriter writer, TenantId value)
        => writer.WriteStringValue(value.Value);

    // For compiled models and precompiled queries
    public override Expression ConstructorExpression
        => Expression.Property(null, InstanceProperty);
}
```

> **Compiled models need Default and Instance**
>
> When generating for NativeAOT (`dotnet ef dbcontext optimize --nativeaot`), the compiled model generator (chapter 21) emits code of the form `TenantIdTypeMapping.Default.Clone(...)` for every type mapping. It requires a **public static property** named `Default` on a **public** type and aborts otherwise. It only takes over converters and readers/writers without extra code when they are *the same instance* as in the default mapping. That is why the mapping uses the static instances rather than `new ...()`. Without NativeAOT, EF 10 resolves type mappings at startup instead; a plugin should still meet the requirements, because users decide on NativeAOT, not the plugin.

Part IV

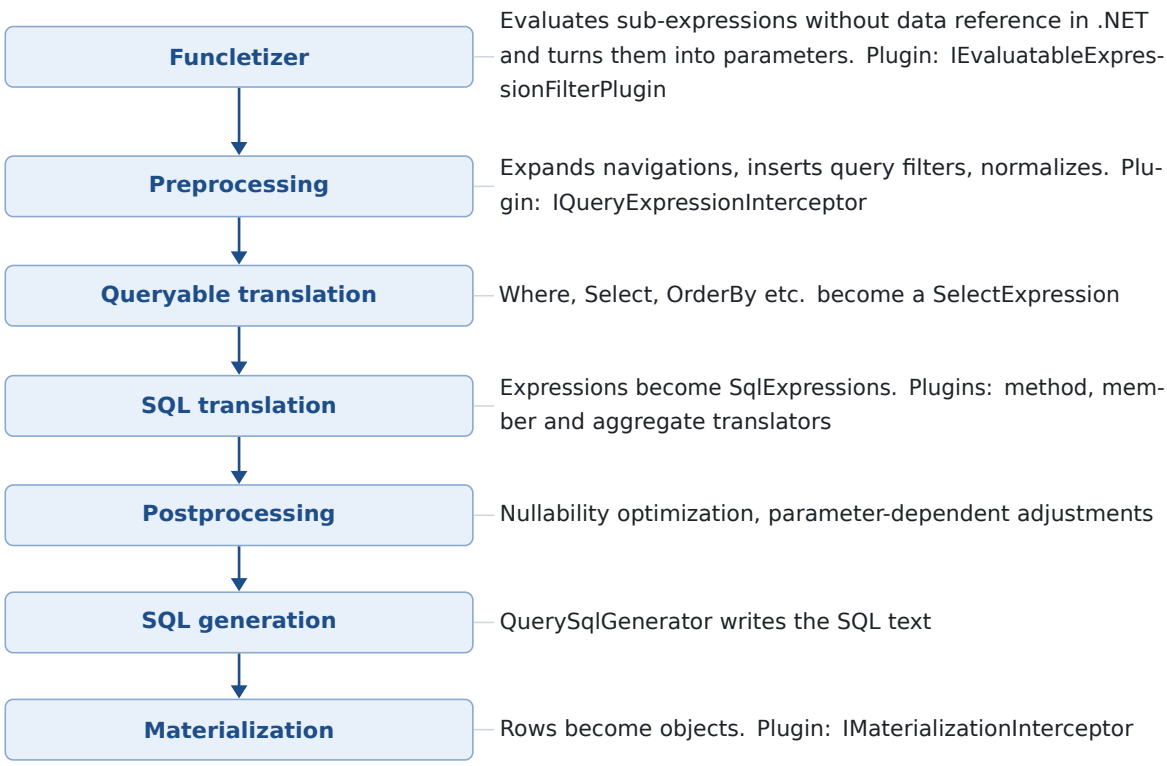
Queries

How LINQ becomes SQL and how a plugin translates its own functions and members.

- Chapter 11: The query pipeline
- Chapter 12: Translating methods and members
- Chapter 13: Preventing client-side evaluation
- Chapter 14: Further query extension points

11 The query pipeline

A LINQ query passes through several stages in EF Core before SQL is produced. Knowing where each stage sits tells you where a plugin has to hook in:



The compiled query is cached. Translators therefore run only once per query shape, not on every execution. Values that change per execution (parameters, session settings) must therefore appear in SQL as parameters or as a database function, never as a constant inserted at translation time. That is exactly why the sample translates `CurrentTenant()` into a call to `current_setting(...)` instead of inserting the current tenant.

12 Translating methods and members

The EF.Functions pattern

Database functions without a .NET counterpart are declared as extension methods on `DbFunctions`. They are never executed; the body only throws an exception in case someone calls them outside of LINQ. The declaration lives in the core package, the translation in the provider package:

TenancyDbFunctionsExtensions.cs (core package)

```
namespace Microsoft.EntityFrameworkCore;

public static class TenancyDbFunctionsExtensions
{
    public static TenantId CurrentTenant(this DbFunctions _)
        => throw new InvalidOperationException(
            $"{nameof(CurrentTenant)} can only be used in LINQ to Entities queries.");
}
```

A method call translator

An `IMethodCallTranslator` receives every method call EF does not know itself. It checks whether it is responsible and then builds a SQL expression tree using `ISqlExpressionFactory`:

Query/CurrentTenantTranslator.cs

```
public class CurrentTenantTranslator : IMethodCallTranslator
{
    private static readonly MethodInfo CurrentTenantMethod =
        typeof(TenancyDbFunctionsExtensions).GetRuntimeMethod(
            nameof(TenancyDbFunctionsExtensions.CurrentTenant), [typeof(DbFunctions)])!;

    private readonly ISqlExpressionFactory _sql;
    private readonly IRelationalTypeMappingSource _typeMappingSource;
    private readonly ITenancySingletonOptions _options;

    public CurrentTenantTranslator(
        ISqlExpressionFactory sql,
        IRelationalTypeMappingSource typeMappingSource,
        ITenancySingletonOptions options)
    {
        // ... ArgumentNullException.ThrowIfNull for all parameters ...
        _sql = sql;
        _typeMappingSource = typeMappingSource;
        _options = options;
    }

    public virtual SqlExpression? Translate(
        SqlExpression? instance, MethodInfo method,
        IReadOnlyList<SqlExpression> arguments,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
    {
        if (method != CurrentTenantMethod)
            return null;                          // not responsible -> next translator
```

```
        var text = _typeMappingSource.FindMapping(typeof(string))!;
        var boolean = _typeMappingSource.FindMapping(typeof(bool))!;

        var currentSetting = _sql.Function(
            "current_setting",
            [
                _sql.Constant(_options.Settings.SettingName, text),
                _sql.Constant(true, boolean),
            ],
            nullable: true,
            argumentsPropagateNullability: [true, false],
            typeof(string), text);

        var nullIfEmpty = _sql.Function(
            "nullif",
            [currentSetting, _sql.Constant(string.Empty, text)],
            nullable: true,
            argumentsPropagateNullability: [true, false],
            typeof(string), text);

        return _sql.Convert(nullIfEmpty, typeof(TenantId), TenantIdTypeMapping.Default);
    }
}
```

This turns `o.TenantId == EF.Functions.CurrentTenant()` into:

Generated SQL (excerpt)

```
WHERE o."TenantId" = CAST(nullif(current_setting('app.tenant_id', TRUE), '') AS uuid)
```

Detail	Meaning
`return null`	Contract for “not responsible”. Never throw for foreign methods; the next translator gets its turn.
`arguments`	For extension methods, `arguments[0]` is the `this` parameter (here `DbFunctions`).
`nullable`	Can the result be NULL? EF uses this to optimize `IS NULL` checks. Wrong values lead to wrong results.
`argumentsPropagateNullability`	Is the result NULL when this argument is NULL? For `nullif(x, '')`: yes for x, no for the empty string.
Type mappings	Every SQL expression needs a mapping. It determines literals, parameters and casts. `Convert` with the TenantId mapping produces `CAST(... AS uuid)`.
`Constant(...)`	Constants become literals via the mapping and are escaped correctly, including the setting name.

A member translator

For properties and fields there is `IMemberTranslator`. The sample translates `tenantId.IsEmpty`. The constant gets the column’s type mapping so that the literal comes out in the right format:

Query/TenantIdMemberTranslator.cs

```
public class TenantIdMemberTranslator : IMemberTranslator
{
    private static readonly PropertyInfo IsEmptyProperty =
        typeof(TenantId).GetProperty(nameof(TenantId.IsEmpty))!;

    private readonly ISqlExpressionFactory _sql;

    public TenantIdMemberTranslator(ISqlExpressionFactory sql)
    {
        ArgumentNullException.ThrowIfNull(sql);

        _sql = sql;
    }

    public virtual SqlExpression? Translate(
        SqlExpression? instance, MemberInfo member, Type returnType,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
    {
        if (instance is null || !Equals(member, IsEmptyProperty))
            return null;

        return _sql.Equal(instance, _sql.Constant(TenantId.Empty, instance.TypeMapping));
    }
}
```

Translator plugins

Translators are not registered directly, but through plugin classes that return a list. The plugins are **scoped** (appendix A) and can therefore inject scoped services such as `ISqlExpressionFactory`:

Query/TenancyTranslatorPlugins.cs

```
public class TenancyMethodCallTranslatorPlugin : IMethodCallTranslatorPlugin
{
    public TenancyMethodCallTranslatorPlugin(
        ISqlExpressionFactory sql,
        IRelationalTypeMappingSource typeMappingSource,
        ITenancySingletonOptions options)
    {
        Translators = [new CurrentTenantTranslator(sql, typeMappingSource, options)];
    }

    public virtual IEnumerable<IMethodCallTranslator> Translators { get; }
}

public class TenancyMemberTranslatorPlugin : IMemberTranslatorPlugin
{
    public TenancyMemberTranslatorPlugin(ISqlExpressionFactory sql)
    {
        Translators = [new TenantIdMemberTranslator(sql)];
    }

    public virtual IEnumerable<IMemberTranslator> Translators { get; }
}
```

> **Tip**
>
> `query.ToQueryString()` produces a query's SQL without executing it and without a database connection. It is the most important tool for developing and testing translators (chapter 23).

13 Preventing client-side evaluation

The first stage of the pipeline, the funcletizer, evaluates every sub-expression in .NET up front that has no reference to query data. `o.Total > limit * 2` thus becomes a parameter. For `EF.Functions.CurrentTenant()` this would be fatal: the expression has no data reference, so EF would execute the method, which throws its exception.

EF knows only a few exceptions. In the source of `EvaluatableExpressionFilter` (EF Core 10) they are `DateTime.Now`, `DateTime.UtcNow`, `DateTime.Today`, their `DateTimeOffset` counterparts, `Guid.NewGuid()`, `Random.Next(...)` and all methods declared in EF's own class `DbFunctionsExtensions`. Custom EF.Functions methods live in a different class and are **not** covered. After that, EF asks all registered plugins:

Core package: TenancyDbFunctionsAttribute.cs, Query/TenancyEvaluatable…Plugin.cs

```
[AttributeUsage(AttributeTargets.Class, Inherited = false)]
public sealed class TenancyDbFunctionsAttribute : Attribute;

public class TenancyEvaluatableExpressionFilterPlugin : IEvaluatableExpressionFilterPlugin
{
    public virtual bool IsEvaluatableExpression(Expression expression)
        => expression is not MethodCallExpression { Method.DeclaringType: { } type }
           || !type.IsDefined(typeof(TenancyDbFunctionsAttribute), inherit: false);
}

[TenancyDbFunctions]
public static class TenancyDbFunctionsExtensions { /* CurrentTenant() */ }
```

The plugin lives in the **core package** and recognizes function classes by an attribute. A single plugin thus also covers the PostgreSQL-specific functions from chapters 37 and 38 and every future provider package, without the core having to know these classes.

> **When the plugin is needed**
>
> Whether a custom EF.Functions method needs this plugin depends on whether it has arguments referencing data. `EF.Functions.Soundex(c.Name)` is harmless because `c.Name` points to a column. Methods without arguments or only with constants (`CurrentTenant()`, `Random()`, `ServerTime()`) always need it. Without it, the exception comes from the method body, which is hard to diagnose.

14 Further query extension points

The sample needs method and member translators. Complex plugins use further points; here is an overview with classification:

Extension point	Purpose	Notes
`IAggregateMethodCallTranslatorPlugin`	Custom aggregate functions in `GroupBy` (e.g. `bool_and`, statistics functions)	Scoped, multiple registrations; implemented in chapter 38
`IQueryExpressionInterceptor`	Rewrite the LINQ expression tree before translation	Singleton interceptor; ideal for query rewrites, tags, hints
`IMaterializationInterceptor`	Create, initialize or check objects during materialization	Singleton interceptor; must be stateless and thread-safe
`IMethodCallTranslatorProvider` etc.	Replace translator infrastructure	Single provider services; only in exceptional cases
`IQuerySqlGeneratorFactory`	SQL generation for custom SqlExpression types	Singleton, provider service; implemented in chapter 37
`IRelationalParameterBasedSqlProcessorFactory`	Nullability processing of custom expression nodes	Scoped, provider service; implemented in chapter 37
`IQueryTranslationPostprocessorFactory`	Adjust the expression tree after translation	Provider service; deep intervention, tightly bound to EF internals

> **Precompiled queries**
>
> With NativeAOT and precompiled queries (since EF Core 9), EF translates queries at build time. Translators then run in the build process, not at run time. They may therefore only depend on options and model, never on runtime state. The sample meets this: the setting name is an option; the tenant itself is only evaluated in the database session.

Part V

Run time

Intervening when saving, connecting and executing, and reporting events properly.

› Chapter 15: Interceptors

› Chapter 16: Diagnostics and logging

15 Interceptors

Interceptors are hooks into EF's run time: before and after commands, connections, transactions, SaveChanges and materialization. They can observe, change values, suppress operations or replace results.

Interface	Events	Lifetime
`ISaveChangesInterceptor`	SavingChanges, SavedChanges, SaveChangesFailed, ThrowingConcurrencyException	Scoped
`IDbConnectionInterceptor`	Opening, closing, creating the connection	Scoped
`IDbCommandInterceptor`	Creating and executing commands, data readers	Scoped
`IDbTransactionInterceptor`	Begin, commit, rollback, savepoints	Scoped
`IIdentityResolutionInterceptor`	Conflicts when attaching already tracked entities	Scoped
`IMaterializationInterceptor`	Creating and initializing entities	**Singleton**
`IQueryExpressionInterceptor`	LINQ expression before compilation	**Singleton**

The two singleton interceptors implement `ISingletonInterceptor`. They are used once per internal provider, not per context, and must therefore be stateless and thread-safe.

Two ways to register

Way	Properties	When
`optionsBuilder.AddInterceptors(...)`	Instances from the application; can use app services; differ per context	Interceptors with app dependencies (tenant, user, clock)
`services.AddScoped<IInterceptor, X>()` in `ApplyServices`	Resolved from the internal provider ("injected interceptors"); have access to EF services, not to app services	Purely technical interceptors without app dependencies

EF combines both ways into one list. The sample uses the first, because both interceptors need the application's tenant (chapter 6).

The SaveChanges interceptor

It sets the tenant of new entities and prevents data of another tenant from being written or reassigned. This complements RLS (defense in depth) and produces clearer errors than the database. Chapter 27 shows which paths bypass it:

`Interceptors/TenantSaveChangesInterceptor.cs (1/2)`

```
public class TenantSaveChangesInterceptor : SaveChangesInterceptor
{
    private readonly ITenantProvider _tenants;

    public TenantSaveChangesInterceptor(ITenantProvider tenants)
    {
```

```
        ArgumentNullException.ThrowIfNull(tenants);

        _tenants = tenants;
    }

    public override InterceptionResult<int> SavingChanges(
        DbContextEventData eventData, InterceptionResult<int> result)
    {
        if (eventData.Context is { } context) ApplyTenant(context);
        return result;
    }

    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result,
        CancellationToken cancellationToken = default)
    {
        if (eventData.Context is { } context) ApplyTenant(context);
        return ValueTask.FromResult(result);
    }
```

Interceptors/TenantSaveChangesInterceptor.cs (2/2, simplified)

```
    internal void ApplyTenant(DbContext context)
    {
        // Which entities are affected is stored in the runtime annotation
        // (chapter "Property-level Fluent API and runtime annotations")
        var changes = context.ChangeTracker.Entries()
            .Where(e => e.State is EntityState.Added or EntityState.Modified
                                or EntityState.Deleted)
            .Select(e => (Entry: e, Name: e.Metadata.GetTenantPropertyName()))
            .Where(x => x.Name is not null)
            .Select(x => (x.Entry, Property: x.Entry.Property(x.Name!)))
            .ToList();
        if (changes.Count == 0) return;

        var current = _tenants.CurrentTenantId
            ?? throw new TenancyException("No active tenant ...");

        foreach (var (entry, property) in changes)
        {
            var currentValue = (TenantId)property.CurrentValue!;
            var originalValue = (TenantId)property.OriginalValue!;
            switch (entry.State)
            {
                case EntityState.Added when currentValue.IsEmpty:
                    property.CurrentValue = current;
                    break;
                case EntityState.Added when currentValue != current:
                    throw Mismatch(entry, currentValue, current);
                case EntityState.Modified when property.IsModified:
                    throw new TenancyException("The tenant property is immutable.");
                case EntityState.Modified or EntityState.Deleted
                    when originalValue != current:
                    throw Mismatch(entry, originalValue, current);
            }
        }
    }
}
```

> **Synchronous and asynchronous**
>
> Always implement **both** variants, synchronous and asynchronous. Depending on `SaveChanges` or `SaveChangesAsync`, EF calls only one of them. An interceptor that only overrides the asynchronous method has no effect on synchronous calls, without anyone noticing.

The logic lives in an `internal` method `ApplyTenant`. Tests can call it directly through `InternalsVisibleTo` without needing a database (chapter 23).

The connection interceptor

It sets the session setting that the policies and `CurrentTenant()` evaluate every time a connection is opened. The flow (intercept opening, sync and async, fetch the tenant) is the same for every database and therefore lives in an abstract base in the core package (chapter 41). The PostgreSQL package only supplies the command. It gets the setting name from the *internal* provider via the context from the event data (`context.GetService<T>()`), and the tenant from the *app* container:

Interceptors/TenantSessionInterceptor.cs (core), Runtime/TenantConnectionInterceptor.cs

```
// Core package
public abstract class TenantSessionInterceptor : DbConnectionInterceptor
{
    protected TenantSessionInterceptor(ITenantProvider tenants)
    {
        ArgumentNullException.ThrowIfNull(tenants);

        Tenants = tenants;
    }

    protected ITenantProvider Tenants { get; }

    public override async Task ConnectionOpenedAsync(
        DbConnection connection, ConnectionEndEventData eventData,
        CancellationToken cancellationToken = default)
    {
        await using var command =
            CreateSessionCommand(connection, eventData.Context, Tenants.CurrentTenantId);
        await command.ExecuteNonQueryAsync(cancellationToken);
    }

    // ConnectionOpened (synchronous) analogous

    protected abstract DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant);
}

// PostgreSQL package
public class TenantConnectionInterceptor : TenantSessionInterceptor
{
    public TenantConnectionInterceptor(ITenantProvider tenants)
        : base(tenants)
    {
    }

    protected override DbCommand CreateSessionCommand(
```

```
        DbConnection connection, DbContext? context, TenantId? tenant)
    {
        var settingName = context?.GetService<ITenancySingletonOptions>()
                              .Settings.SettingName
                          ?? TenancySettings.Default.SettingName;

        var command = connection.CreateCommand();
        command.CommandText = "SELECT set_config(@name, @value, false)";
        AddParameter(command, "name", settingName);
        // Always set, even empty: no stale value remains
        AddParameter(command, "value", tenant?.ToString() ?? string.Empty);
        return command;
    }
}
```

> **Empty string instead of NULL**
>
> Npgsql resets the session state when a connection returns to the pool. After that, `current_setting('app.tenant_id', true)` returns an empty string instead of `NULL`. A `''::uuid` would abort every query. Policy and translator therefore use `nullif(..., '')`: without a tenant the value is `NULL`, the comparison is never true and no rows are visible. That is the safe direction.

By default EF opens and closes the connection for every query. Every opening costs an extra round trip here. With many small queries it pays off to open the connection once per request explicitly (chapter 34).

Where the connection interceptor does not run

The interceptor runs when EF opens a connection, and only then. Two situations bypass it, and both are easy to create by accident:

Situation	Consequence	Remedy
An already open connection is passed in (`UseNpgsql(connection)`, `SetDbConnection`)	EF never opens it, so the interceptor never runs. The session keeps whatever value it has: none, or that of a previous tenant	Pass connections closed and let EF open them; otherwise call `Database.ApplyTenantSession()`
The connection stays open (`OpenConnection`, an open transaction) while the tenant changes	The new tenant only takes effect when the connection is opened again	One context, one tenant; if that is not possible, call `Database.ApplyTenantSession()` after the change

tests/...IntegrationTests/ConnectionLifetimeTests.cs (shortened)

```
// Already open, with a session value left over from earlier work (tenant B)
await using var connection = new NpgsqlConnection(appConnectionString);
await connection.OpenAsync();
// ... SELECT set_config('app.tenant_id', <tenant B>, false) ...

// Meant for tenant A, but EF does not open the connection: the interceptor never runs
await using var db = CreateAppContext(connection, tenantA);
var orders = await db.Orders.ToListAsync();       // the rows of tenant B!
```

For these cases the plugin offers the explicit counterpart to the interceptor. `Database.ApplyTenantSession()` writes the current tenant into the session if the connection is open, and does nothing if it is closed, because EF will then open it and the interceptor takes over. It reuses the interceptor's own command, so it works for every provider package:

Usage

```
await using var db = CreateAppContext(connection, tenantA);   // connection already open
await db.Database.ApplyTenantSessionAsync();                  // session now: tenant A
var orders = await db.Orders.ToListAsync();                   // the rows of tenant A
```

Interceptors/TenantSessionInterceptor.cs (excerpt)

```
public virtual async Task<bool> ApplyToOpenConnectionAsync(
    DbContext context, CancellationToken cancellationToken = default)
{
    var connection = context.Database.GetDbConnection();
    if (connection.State != ConnectionState.Open)
        return false;                    // EF opens it later; the interceptor takes over

    await using var command =
        CreateSessionCommand(connection, context, Tenants.CurrentTenantId);
    // Inside an explicit transaction the command must be enlisted in it
    command.Transaction = context.Database.CurrentTransaction?.GetDbTransaction();
    await command.ExecuteNonQueryAsync(cancellationToken);
    return true;
}
```

The extension method `ApplyTenantSession` in the core package finds the interceptor among the context's interceptors and fails with a clear message if there is none. Two details matter: the method needs the context, not just the connection, because the provider reads its settings from the internal service provider; that is why `CreateSessionCommand` receives the `DbContext` instead of the event data. And the command must join a running transaction: PostgreSQL is lenient here, SQL Server rejects a command that is not enlisted. The SQL Server tests check exactly this case; without the line they fail.

> **One context, one tenant**
>
> Isolation follows the connection, not the context. A context used for several tenants, or a connection that outlives a request, carries the tenant of the moment it was opened. `ConnectionLifetimeTests` shows both cases against a real database, together with the two remedies: a closed connection that EF opens itself, and `ApplyTenantSession()`. A plugin whose hooks cannot cover every path should offer such an explicit API instead of only documenting the gap.

16 Diagnostics and logging

A professional plugin reports events through EF's logging system. Messages then end up in the same channels users configure anyway (`ILogger`, `LogTo`, `DiagnosticSource`), and warnings can be ignored, logged or turned into errors via `ConfigureWarnings`.

Event IDs

Diagnostics/TenancyEventId.cs (shortened)

```
public static class TenancyEventId
{
    public const int TenancyBaseId = 1_150_000;     // own, documented range

    private enum Id
    {
        TenantIsolationApplied = TenancyBaseId,
        TenantIsolationOptedOut,
        PoliciesWithoutRowLevelSecurity,
    }

    private static readonly string ModelPrefix = "Acme.EntityFrameworkCore.Tenancy.Model.";

    public static readonly EventId TenantIsolationApplied =
        Make(Id.TenantIsolationApplied);
    public static readonly EventId TenantIsolationOptedOut =
        Make(Id.TenantIsolationOptedOut);
    // ...

    private static EventId Make(Id id) => new((int)id, ModelPrefix + id);
}
```

EF and providers occupy low ranges. A plugin chooses its own high range and documents it. Names follow EF's scheme "Assembly.Category.Event" so that prefix filters work.

Event definitions

An `EventDefinition<T>` bundles event ID, log level, message template and the warning behavior from the options. EF caches its definitions per internal provider; the plugin does the same with its own class registered as a singleton (chapter 5):

Diagnostics/TenancyLoggingDefinitions.cs (excerpt)

```
public class TenancyLoggingDefinitions
{
    private EventDefinition<string>? _tenantIsolationOptedOut;

    public virtual EventDefinition<string> LogTenantIsolationOptedOut(
        IDiagnosticsLogger logger)
        => LazyInitializer.EnsureInitialized(ref _tenantIsolationOptedOut, () =>
            Create(logger, TenancyEventId.TenantIsolationOptedOut, LogLevel.Warning,
                nameof(TenancyEventId.TenantIsolationOptedOut),
                "Entity '{entityType}' implements ITenantOwned but is excluded from "
```

```
                + "tenant isolation."));

    private static EventDefinition<string> Create(
        IDiagnosticsLogger logger, EventId eventId, LogLevel level,
        string name, string message)
        => new(
            logger.Options,                      // contains the warning behavior
            eventId,
            level,
            "TenancyEventId." + name,            // appears in warning-as-error messages
            l => LoggerMessage.Define<string>(l, eventId, message));
}
```

The logger extension

The actual log method follows the pattern of EF's `CoreLoggerExtensions` exactly: classic logging first, then the event data for DiagnosticSource and `LogTo`:

Diagnostics/TenancyLoggerExtensions.cs (excerpt)

```
private static void LogEntityType(
    IDiagnosticsLogger<DbLoggerCategory.Model> diagnostics,
    EventDefinition<string> definition,
    IReadOnlyEntityType entityType)
{
    // 1. ILogger; throws with ConfigureWarnings(w => w.Throw(...))
    if (diagnostics.ShouldLog(definition))
        definition.Log(diagnostics, entityType.DisplayName());

    // 2. DiagnosticSource and LogTo (simple logging)
    if (diagnostics.NeedsEventData(definition, out var diagnosticSource, out var simpleLog))
    {
        var eventData = new EntityTypeEventData(definition, GenerateMessage, entityType);
        diagnostics.DispatchEventData(definition, eventData, diagnosticSource, simpleLog);
    }
}
```

Users can then turn the warning into an error:

Usage

```
options.UseNpgsql(cs)
       .UseAcmeTenancy()
       .ConfigureWarnings(w => w.Throw(TenancyEventId.TenantIsolationOptedOut));

// -> InvalidOperationException while building the model:
//    "An error was generated for warning '...TenantIsolationOptedOut': Entity
//     'AuditEntry' ... This exception can be suppressed or logged by passing event ID
//     'TenancyEventId.TenantIsolationOptedOut' to the 'ConfigureWarnings' method ..."
```

> **Tip**
>
> Security-relevant deviations belong in the log as a **warning**, not as a debug message. Teams with strict requirements escalate them to errors via `ConfigureWarnings`; others at least see them at startup. The plugin does not have to make this decision for everyone.

Part VI

Migrations and design time

Generating schema changes, extending the dotnet ef tooling and supporting compiled models.

› Chapter 17: How migrations work

› Chapter 18: Declarative SQL from annotations

› Chapter 19: Custom migration operations

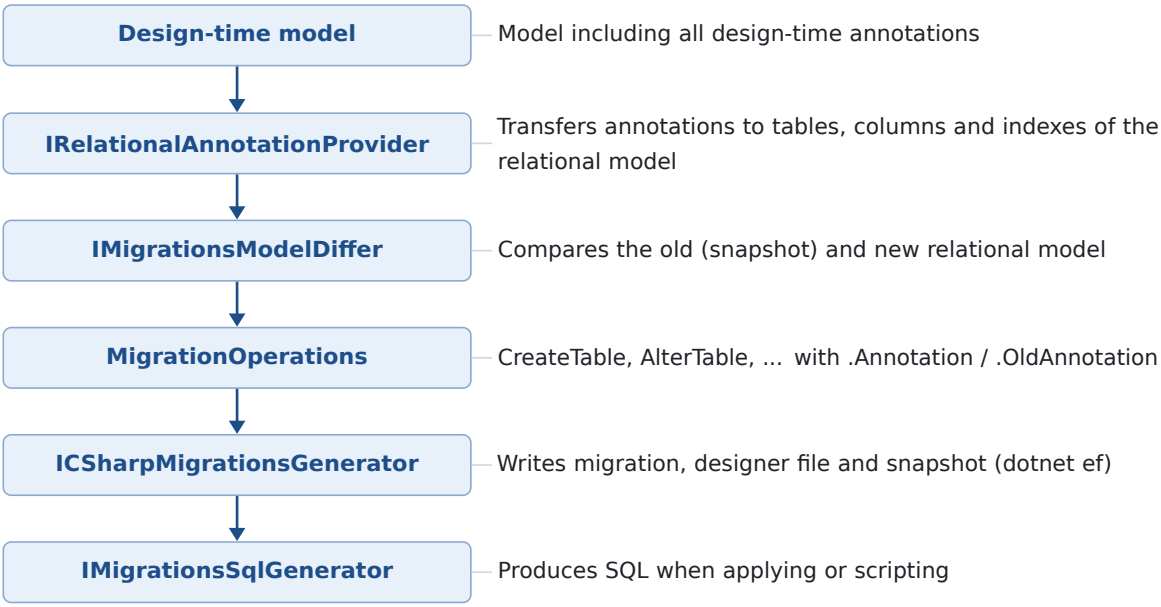
› Chapter 20: Design-time services

› Chapter 21: Compiled models, trimming and NativeAOT

› Chapter 22: Scaffolding (reverse engineering)

17 How migrations work

Migrations compare two models and produce operations and SQL from them. A plugin can hook in at several points; to choose the right one you need to know the flow:



There are two basic strategies for plugins. **Declarative**: the plugin describes the desired state as annotations; EF detects changes by itself, and the plugin only has to produce SQL. **Imperative**: the plugin offers its own operations that users write into migrations by hand. Good plugins use both: declarative for everything that belongs to the model, imperative for everything else.

> **Note**
>
> `Database.EnsureCreated()` uses the same pipeline: it computes the diff from the empty state to the design-time model and executes the SQL of the migrations generator. A plugin's custom migration SQL therefore takes effect there too, which simplifies integration tests considerably (chapter 23).

18 Declarative SQL from annotations

The annotation provider

The differ compares the *relational* model (tables, columns), not the entity types. Which annotations a table carries is decided by the `IRelationalAnnotationProvider`. There is one per context, and it belongs to the provider; the plugin therefore derives from the Npgsql implementation:

Metadata/Internal/TenancyAnnotationProvider.cs

```
public class TenancyAnnotationProvider : NpgsqlAnnotationProvider
{
    public TenancyAnnotationProvider(RelationalAnnotationProviderDependencies dependencies)
        : base(dependencies)
    {
    }

    public override IEnumerable<IAnnotation> For(ITable table, bool designTime)
    {
        foreach (var annotation in base.For(table, designTime))
            yield return annotation;

        if (!designTime)          // migrations only: the runtime model stays lean
            yield break;

        var entityType = (IEntityType)table.EntityTypeMappings.First().TypeBase;
        foreach (var annotation in entityType.GetAnnotations())
        {
            if (!annotation.Name.StartsWith(RlsAnnotationNames.Prefix,
                                            StringComparison.Ordinal))
                continue;
            // Do not pass on the opt-out marker, otherwise empty migrations appear
            if (annotation.Name == RlsAnnotationNames.Enabled && annotation.Value is false)
                continue;
            yield return annotation;
        }
    }
}
```

From here EF takes over: when a policy changes, the differ produces an `AlterTableOperation` with new and old annotations. In the generated migration it looks like this, without any custom code generator:

Generated migration (excerpt)

```
migrationBuilder.AlterTable(name: "Orders")
    .Annotation("Acme:Rls:Policy:tenant_isolation", "{...new...}")
    .OldAnnotation("Acme:Rls:Policy:tenant_isolation", "{...old...}");
```

The SQL generator

There is also only one `IMigrationsSqlGenerator`. The plugin derives from the Npgsql generator and extends `CreateTable` and `AlterTable`:

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (excerpt)

```
protected override void Generate(
    CreateTableOperation operation, IModel? model,
    MigrationCommandListBuilder builder, bool terminate = true)
{
    // Unterminated SQL cannot take further statements (as with Npgsql comments)
    if (!terminate && operation.HasRlsConfiguration())
        throw new ArgumentException("RLS cannot be emitted in an unterminated CREATE TABLE.");

    base.Generate(operation, model, builder, terminate);
    if (terminate)
        GenerateRowLevelSecurity(operation.Name, operation.Schema, previous: null,
            operation, builder);
}

protected override void Generate(
    AlterTableOperation operation, IModel? model, MigrationCommandListBuilder builder)
{
    base.Generate(operation, model, builder);
    GenerateRowLevelSecurity(operation.Name, operation.Schema, operation.OldTable,
        operation, builder);
}
```

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (shortened)

```
protected virtual void GenerateRowLevelSecurity(string tableName, string? schema,
    IReadOnlyAnnotatable? previous, IReadOnlyAnnotatable current,
    MigrationCommandListBuilder builder)
{
    var table = Delimit(tableName, schema);
    var oldPolicies = previous?.GetRlsPolicies() ?? new Dictionary<string, string>();
    var newPolicies = current.GetRlsPolicies();

    // 1. Drop removed or changed policies
    foreach (var (name, json) in oldPolicies)
        if (!newPolicies.TryGetValue(name, out var n) || n != json)
            Statement(builder, $"DROP POLICY {Delimit(name)} ON {table}");

    // 2. RLS state (ENABLE/DISABLE, FORCE/NO FORCE)
    if ((previous?.IsRlsEnabled() ?? false) != current.IsRlsEnabled())
        Statement(builder, $"ALTER TABLE {table} "
            + (current.IsRlsEnabled() ? "ENABLE" : "DISABLE") + " ROW LEVEL SECURITY");
    // ... FORCE analogous

    // 3. Create new or changed policies
    foreach (var (name, json) in newPolicies)
        if (!oldPolicies.TryGetValue(name, out var o) || o != json)
            Statement(builder, CreatePolicySql(table, RlsPolicy.Deserialize(json)));
}

protected virtual void Statement(MigrationCommandListBuilder builder, string sql)
{
    builder.Append(sql).AppendLine(Dependencies.SqlGenerationHelper.StatementTerminator);
    EndStatement(builder);
}
```

Detail	Reason
DROP + CREATE instead of `ALTER POLICY`	`ALTER POLICY` can change neither `FOR` nor PERMISSIVE/RESTRICTIVE. DDL is transactional in PostgreSQL; no visible gap occurs.
`EndStatement(builder)`	Ends the command like the base class does, honoring transaction suppression.
Own `Delimit` helpers	Npgsql's `DelimitIdentifier` helpers are private. Always escape identifiers via `Dependencies.SqlGenerationHelper`, never by string concatenation.
DropTable, RenameTable	Nothing to do: PostgreSQL removes or moves policies together with the table.

19 Custom migration operations

Some things do not belong in the model but in migrations: roles, grants, extensions, data backfills. For these the plugin offers its own operations. They consist of three parts: the operation class, an extension method on `MigrationBuilder` and the SQL generation in the generator.

Migrations/Operations/TenancyOperations.cs (excerpt)

```
public class CreateApplicationRoleOperation : MigrationOperation
{
    public required string Name { get; init; }
    public bool CanLogin { get; init; }       // passwords do NOT belong in migrations
}
```

Migrations/Operations/TenancyMigrationBuilderExtensions.cs (excerpt)

```
namespace Microsoft.EntityFrameworkCore.Migrations;      // always imported in migrations

public static class TenancyMigrationBuilderExtensions
{
    public static OperationBuilder<CreateApplicationRoleOperation> CreateApplicationRole(
        this MigrationBuilder migrationBuilder, string name, bool canLogin = false)
    {
        var operation = new CreateApplicationRoleOperation
        {
            Name = TenancyIdentifiers.ValidateIdentifier(name, "role name"),
            CanLogin = canLogin,
        };
        migrationBuilder.Operations.Add(operation);
        return new OperationBuilder<CreateApplicationRoleOperation>(operation);
    }
}
```

The generator dispatches operations through the central `Generate(MigrationOperation, ...)` method. Unknown types go to the base class:

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (excerpt)

```
protected override void Generate(
    MigrationOperation operation, IModel? model, MigrationCommandListBuilder builder)
{
    switch (operation)
    {
        case CreateApplicationRoleOperation createRole: Generate(createRole, builder); break;
        case DropApplicationRoleOperation dropRole:     Generate(dropRole, builder); break;
        case GrantTablePrivilegesOperation grant:       Generate(grant, builder); break;
        default: base.Generate(operation, model, builder); break;
    }
}

protected virtual void Generate(CreateApplicationRoleOperation operation,
    MigrationCommandListBuilder builder)
{
    var literal = Literal(operation.Name);           // 'shop_app'  (for the comparison)
    var identifier = Delimit(operation.Name);    // shop_app (CREATE ROLE, "..." if needed)
    var login = operation.CanLogin ? "LOGIN" : "NOLOGIN";
```

```
        builder
            .AppendLine("DO $$")
            .AppendLine("BEGIN")
            .AppendLine("    IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles")
            .AppendLine($"                   WHERE rolname = {literal}) THEN")
            .AppendLine($"        CREATE ROLE {identifier} {login} NOSUPERUSER NOBYPASSRLS;")
            .AppendLine("    END IF;")
            .AppendLine("END")
            .Append("$$");
        builder.AppendLine(Dependencies.SqlGenerationHelper.StatementTerminator);
        EndStatement(builder);
    }
```

Literal or identifier

The role name appears twice, but in two forms: as a **string literal** in the comparison (`'shop_app'`, produced by the string type mapping) and as an **identifier** in `CREATE ROLE` (`shop_app`, in double quotes if it needs them). Mixing them up or building them by string concatenation produces invalid SQL or an injection hole. The extension method additionally validates the name.

Tip

Write operations idempotently (`IF NOT EXISTS`, `DROP ... IF EXISTS`). Roles exist cluster-wide and outlive databases; a migration that fails on its second run against a new database is a common annoyance.

In a migration, the operations are used like EF's own:

`Migrations/20261021_Initial.cs (hand-edited)`

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // BEFORE the tables (chapter "Advanced migrations")
    migrationBuilder.CreateApplicationRole("shop_support");
    // ... operations generated by EF (tables, RLS via annotations) ...
    migrationBuilder.CreateApplicationRole("shop_app", canLogin: true);
    migrationBuilder.GrantTablePrivileges("shop_app");
}

protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropApplicationRole("shop_app");
    // ...
}
```

When the differ should produce custom operations

If the *differ* should produce custom operations automatically, you need a derived `IMigrationsModelDiffer` and an `ICSharpMigrationOperationGenerator` that writes the operation as C#. Both are single services with a lot of internal API. The declarative route via annotations (chapter 18) is almost always the better choice.

20 Design-time services

The `dotnet ef` tools (adding migrations, writing snapshots, scaffolding, compiled models) use their own service provider with design-time services. A plugin extends it through `IDesignTimeServices`.

Loaded automatically

So that users do not need to configure anything, an assembly attribute points to the class. The tooling finds it in every assembly the startup project references. The second argument restricts loading to contexts with the matching provider:

Properties/AssemblyInfo.cs

```
[assembly: DesignTimeServicesReference(
    "Acme.EntityFrameworkCore.Tenancy.PostgreSQL.Design.TenancyDesignTimeServices, "
        + "Acme.EntityFrameworkCore.Tenancy.PostgreSQL",
    "Npgsql.EntityFrameworkCore.PostgreSQL")]
```

Design/TenancyDesignTimeServices.cs

```
public class TenancyDesignTimeServices : IDesignTimeServices
{
    public void ConfigureDesignTimeServices(IServiceCollection services)
    {
        // Remove all registrations (EF default, Npgsql), then add ours:
        // independent of the order in which the tooling loads services
        services.RemoveAll<IAnnotationCodeGenerator>();
        services.AddSingleton<IAnnotationCodeGenerator, TenancyAnnotationCodeGenerator>();
    }
}
```

Readable snapshots: the annotation code generator

By default EF writes unknown annotations as `HasAnnotation(...)` into the snapshot, for policies with JSON values. That works, but is hardly readable. A custom `IAnnotationCodeGenerator` translates them into the Fluent API:

ShopContextModelSnapshot.cs

```
// Before
b.HasAnnotation("Acme:Rls:Enabled", true);
b.HasAnnotation("Acme:Rls:Policy:tenant_isolation", "{\"Name\":\"tenant_isolation\",...}");

// After
RlsEntityTypeBuilderExtensions.HasRowLevelSecurity(b, true);
RlsEntityTypeBuilderExtensions.HasPolicy(b, "tenant_isolation", p => p
    .Using("\"TenantId\" = nullif(current_setting('app.tenant_id', true), '')::uuid")
    .WithCheck("\"TenantId\" = nullif(current_setting('app.tenant_id', true), '')::uuid"));
```

EF Core 10 writes extension methods in the snapshot as static calls (`Extensions.Method(b, ...)`) instead of `b.Method(...)`. This avoids ambiguities when several providers or plugins define methods with the same name. Tests that check the generated code should expect this form.

Design/Internal/TenancyAnnotationCodeGenerator.cs (excerpt)

```
public override IReadOnlyList<MethodCallCodeFragment> GenerateFluentApiCalls(
    IEntityType entityType, IDictionary<string, IAnnotation> annotations)
{
    var calls = new List<MethodCallCodeFragment>();

    // CONTRACT: REMOVE translated annotations from the dictionary
    if (annotations.TryGetValue(RlsAnnotationNames.Enabled, out var enabled)
        && enabled.Value is true)
    {
        annotations.Remove(RlsAnnotationNames.Enabled);
        var forced = annotations.Remove(RlsAnnotationNames.Forced, out var f)
                     && f.Value is true;
        calls.Add(new MethodCallCodeFragment(HasRowLevelSecurityMethod, forced));
    }

    foreach (var key in annotations.Keys
                 .Where(k => k.StartsWith(RlsAnnotationNames.PolicyPrefix,
                                          StringComparison.Ordinal))
                 .ToList())                          // copy: the dictionary changes
    {
        var policy = RlsPolicy.Deserialize((string)annotations[key].Value!);
        annotations.Remove(key);
        calls.Add(new MethodCallCodeFragment(HasPolicyMethod, policy.Name,
            new NestedClosureCodeFragment("p", BuildPolicyChain(policy))));
    }

    calls.AddRange(base.GenerateFluentApiCalls(entityType, annotations));
    return calls;
}
```

Pitfall	Solution
Translated annotation stays in the dictionary	Appears twice in the snapshot. Always remove it; whatever remains, EF writes as `HasAnnotation`.
Generic method in the `MethodInfo`	The snapshot uses the non-generic `EntityTypeBuilder`. Pick the non-generic overload via `GetRuntimeMethod` with exact parameter types.
`params` argument as an array	`MethodCallCodeFragment` takes `params object?[]`. A `string[]` would become the argument list through array covariance: wrap it in `new object?[] { roles }`.
Enums in the snapshot	Rendered as `PolicyCommand.Select`. The enum therefore lives in the always-imported namespace `Microsoft.EntityFrameworkCore`.
Lambda arguments	`NestedClosureCodeFragment("p", chain)` produces `p => p.Using(...)`; chains are built with `fragment.Chain(...)`.

The generator only affects the snapshot and designer files. The actual migrations keep `.Annotation(...)` on `AlterTable`; that is the intended mechanism for changes.

21 Compiled models, trimming and NativeAOT

`dotnet ef dbcontext optimize` produces a **compiled model**: C# code that builds the runtime model directly instead of computing it at startup through conventions. This shortens startup for large models considerably and is a prerequisite for NativeAOT. A plugin must be prepared for it in four places:

Building block	Requirement	In the sample
Type mappings	Public type with a public static property `Default` (used for NativeAOT); converters/comparers as shared instances	`TenantIdTypeMapping.Default`, `TenantIdConverter.Instance`
JSON reader/writer	Static `Instance` and `ConstructorExpression`	`TenantIdJsonReaderWriter`
Value generators	Configured with `HasValueGeneratorFactory(Type)`; factory with a public parameterless constructor	`TenantIdValueGeneratorFactory` (chapter 28)
Annotations	Simple values are taken over by EF automatically; an `ICSharpRuntimeAnnotationCodeGenerator` filters design-time annotations and translates complex values	Filters RLS policies, keeps the runtime annotation (chapter 39)

The third point is often misunderstood. EF takes annotations with simple values (strings, numbers, bool, enums) into the compiled model **automatically**; the source of `CSharpRuntimeAnnotationCodeGenerator` shows this, writing them via `GenerateSimpleAnnotations`. A custom generator is therefore not needed to *keep* annotations, but to *filter out* pure design-time annotations that nobody reads at run time, and to translate annotations with complex values into code. Chapter 39 shows both in the sample.

Trimming and AOT

Both packages set `<IsAotCompatible>true</IsAotCompatible>`. The compiler then reports code that uses reflection without annotations. The JSON serialization of the policies uses a source generator instead of reflection:

Metadata/RlsPolicy.cs (excerpt)

```
[JsonSourceGenerationOptions(UseStringEnumConverter = true)]
[JsonSerializable(typeof(RlsPolicy))]
internal partial class RlsJsonContext : JsonSerializerContext;
```

> **Note**
>
> Conventions, snapshots and migrations only run at design time or during model building; with a compiled model and precompiled queries they disappear at run time. For AOT, the critical parts are therefore type mappings, converters, translators (run at build time) and interceptors.

22 Scaffolding (reverse engineering)

`dotnet ef dbcontext scaffold` reads an existing database schema and generates entities and a context. A plugin that manages its own schema objects can recognize them during this too. The route goes through two design-time services:

Service	Task
`IDatabaseModelFactory`	Reads the schema into a `DatabaseModel`. A derived Npgsql factory can query `pg_policies` and `pg_class.relrowsecurity` and attach them as annotations to `DatabaseTable`.
`IAnnotationCodeGenerator`	Writes these annotations as Fluent API into the generated `OnModelCreating`, with the same logic as for the snapshot.

The sample implements both: a custom database model factory reads RLS state and policies from the database, and the annotation code generator from chapter 20 writes them into the scaffolded context. Chapter 40 shows the implementation in detail.

Part VII

Quality and delivery

Testing, versioning, packaging, and the checklist from A to Z.

› Chapter 23: Test strategy

› Chapter 24: Versioning and compatibility

› Chapter 25: Packaging and documentation

› Chapter 26: The checklist from A to Z

23 Test strategy

Plugins can be tested to a surprising extent without a database. Model building, type mappings, SQL translation, migration SQL and snapshots only need a connection string, no connection. The sample's test strategy has four levels:

Level	Checks	Database
Model tests	Annotations, conventions, validation, options, caching	no
SQL tests	Translations via `ToQueryString`, migration SQL via differ and generator	no
Snapshot tests	Code generator as a string and as a round trip (Roslyn)	no
Integration tests	Real behavior: RLS, roles, interceptors (Testcontainers)	yes (Docker)

Test helpers

Tests build many contexts with different `OnModelCreating`. Because EF caches the model per internal provider, every test must get its own provider; that is what `EnableServiceProviderCaching(false)` is for:

tests/.../TestModel.cs (shortened)

```
public static TestContext Create(
    Action<TenancyOptionsBuilder>? tenancy = null,
    Action<ModelBuilder>? model = null,
    bool cacheServiceProvider = false,
    Action<DbContextOptionsBuilder>? configure = null)
{
    var builder = new DbContextOptionsBuilder<TestContext>()
        .UseNpgsql("Host=localhost;Database=unused")       // no connection needed
        .UseAcmeTenancy(tenancy)
        .EnableServiceProviderCaching(cacheServiceProvider);
    configure?.Invoke(builder);
    return new TestContext(builder.Options, model);
}

public static IReadOnlyList<string> MigrationSql(DbContext? source, DbContext target)
{
    var operations = target.GetService<IMigrationsModelDiffer>().GetDifferences(
        source?.GetService<IDesignTimeModel>().Model.GetRelationalModel(),
        target.GetService<IDesignTimeModel>().Model.GetRelationalModel());

    return target.GetService<IMigrationsSqlGenerator>()
        .Generate(operations, target.GetService<IDesignTimeModel>().Model)
        .Select(c => c.CommandText.Trim()).ToList();
}
```

Use the design-time model

Always use the **design-time model** for migration tests (`IDesignTimeModel`). `context.Model` is the lean runtime model; annotations that the annotation provider only passes on at design time are missing from it.

> **Why not SQLite or InMemory?**
>
> Tests against SQLite or the InMemory provider are tempting because they need no Docker. For a plugin they prove little: InMemory is not relational, so type mappings, translators and migrations never run; SQLite has a different SQL dialect, other type mappings and no Row Level Security. A provider-specific plugin is tested with its own provider: without a database (connection string only) for everything up to the SQL text, and against the real database for behavior.

What is tested

Test	Ensures
Caching with different options	Two contexts with different settings get different models (hash mistake from chapter 4).
Invalid setting names	Validation including an injection attempt.
Type mapping without configuration	`TenantId` becomes `uuid`; literal format; a different column type is not claimed.
`CurrentTenant()` via `ToQueryString`	Translation, and that nothing is evaluated client-side (chapter 13).
Explicit beats convention	An own policy with the same name is kept; the Fluent API overrides the attribute.
Warning as error	The opt-out warning becomes an error via `ConfigureWarnings`.
Validation	RLS on a derived type and invalid policies throw while building the model.
No phantom migration	An unchanged model produces not a single operation.
Migration SQL	Initial, change (DROP + CREATE), opt-out, custom operations.
SaveChanges logic	Setting, foreign tenant, reassignment, no tenant.
Snapshot	Fluent API instead of annotations; round trip without differences.
RLS in PostgreSQL	Isolation, no rows without a tenant, WITH CHECK applies.
Edge cases (chapters 27–32)	ExecuteUpdate and raw SQL against RLS, value generation, JSON columns, compiled model, idempotent script.
Composability	Decorator around the migrations generator (chapter 5), EFCore.NamingConventions in both orders (chapter 8).
Annotation format	Serialized policies stay identical; tolerant reading (chapter 24).
Context lifetimes	DbContext factory, background jobs, open connections (chapters 15, 35).
Cross-tenant access	Reporting role reads all tenants but cannot write; the app role cannot switch RLS off (chapter 33).

The snapshot round trip

The most valuable test for design-time code: generate the snapshot, compile it with Roslyn, build the model from it and diff it against the original. Any discrepancy between code generator and Fluent API would otherwise produce a phantom migration for every user:

`tests/.../SnapshotTests.cs (excerpt)`

```
[Fact]
public void Snapshot_roundtrip_produces_no_differences()
```

```
{
    using var ctx = CreateContext();
    var snapshot = Compile(GenerateSnapshot(ctx));

    // As in EF's migrator: finalize and initialize
    var model = snapshot.Model;
    if (model is IMutableModel mutable) model = mutable.FinalizeModel();
    model = ctx.GetService<IModelRuntimeInitializer>()
        .Initialize(model, designTime: true, validationLogger: null);

    var differences = ctx.GetService<IMigrationsModelDiffer>().GetDifferences(
        model.GetRelationalModel(), ctx.DesignModel().GetRelationalModel());

    Assert.Empty(differences);
}
```

Integration tests with Testcontainers

The real security behavior can only be verified against PostgreSQL. The fixture starts a container, creates the schema with `EnsureCreated` (including RLS, chapter 17), executes the custom migration operations for real and connects the tests with a restricted role:

tests/...IntegrationTests/PostgresFixture.cs (excerpt)

```
await owner.Database.EnsureCreatedAsync();          // RLS, policies, index

var commands = owner.GetService<IMigrationsSqlGenerator>().Generate(
[
    new CreateApplicationRoleOperation { Name = AppRole, CanLogin = true },
    new GrantTablePrivilegesOperation { Role = AppRole },
]);
await owner.GetService<IMigrationCommandExecutor>().ExecuteNonQueryAsync(
    commands, owner.GetService<IRelationalConnection>(),
    new MigrationExecutionState(), commitTransaction: true);
```

tests/...IntegrationTests/RowLevelSecurityTests.cs (excerpt)

```
[Fact]
public async Task Database_rejects_rows_of_other_tenant()
{
    // Bypass the interceptor: direct SQL. The policy's WITH CHECK must apply.
    var tenants = new MutableTenantProvider { CurrentTenantId = TenantId.New() };
    await using var db = _fixture.CreateAppContext(tenants);

    var ex = await Assert.ThrowsAsync<PostgresException>(() => db.Database.ExecuteSqlAsync(
        $"INSERT INTO \"Orders\" (\"TenantId\", \"Total\") VALUES ({Guid.NewGuid()}, 1)"));

    Assert.Equal(PostgresErrorCodes.InsufficientPrivilege, ex.SqlState);
}
```

> **Tip**
>
> For providers and very deep plugins, EF Core provides the **specification tests** (`Microsoft.EntityFrameworkCore.Relational.Specification.Tests`). This is the same test suite EF and the providers themselves are tested against. The NetTopologySuite plugin in the EF repository shows how a plugin uses it.

24 Versioning and compatibility

Align versions with EF Core

The plugin's major version follows EF Core's: version 10.x for EF Core 10. Users immediately see which combination fits, and the plugin can adopt breaking changes with every EF major version. Dependencies are limited to this major version:

Directory.Packages.props (excerpt)

```
<PackageVersion Include="Microsoft.EntityFrameworkCore" Version="[10.0.0,11.0.0)" />
<PackageVersion Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="[10.0.0,11.0.0)" />
```

Dealing with internal APIs

Plugins rarely get by without internal APIs: the sample derives from `NpgsqlMigrationsSqlGenerator`, `NpgsqlAnnotationProvider` and `NpgsqlAnnotationCodeGenerator` and uses `NpgsqlOptionsExtension`. Such types can change even in minor versions. EF reports their use with warning **EF1001**.

Measure	Effect
Look for a public alternative	Every internal API avoided is one risk less.
Bundle internal access	A few classes encapsulate it; a break affects one place.
Set `<NoWarn>EF1001</NoWarn>` deliberately	Project-wide and documented, not scattered per line.
Limit the version range	Prevents users from getting untested combinations.
CI against previews	A build against the next EF preview shows breaks months earlier.
Round trip and SQL tests	Detect behavior changes the compiler does not see.

> **Multi-targeting**
>
> Supporting several EF major versions at once is possible through multi-targeting, but doubles test effort and conditional compilation. For most plugins, one release branch per EF major version is simpler.

Evolving annotation formats

Everything a plugin writes into annotations ends up in its users' model snapshots, often long before the next upgrade. If a new plugin version serializes the same configuration differently (an additional JSON property, a different order, another enum spelling), the differ compares two different strings. Every user then gets a migration that drops and recreates every policy without any actual change. The serialized format is therefore part of the public API, just like the methods.

Rule	How
Pin the format with a test	A golden-value test compares the serialized form character by character.

Rule	How
Read tolerantly	Missing properties become defaults, unknown ones are ignored (the default behavior of System.Text.Json).
Only add, and do not write defaults	New optional properties with `[JsonIgnore(Condition = WhenWritingDefault)]` keep existing annotations identical.
Real format changes only with a major version	Document them, ideally together with the EF major upgrade, when users expect migrations anyway.

tests/.../AnnotationFormatTests.cs (excerpt)

```
[Fact]
public void Serialized_policy_format_is_stable()
{
    var policy = new RlsPolicy("support_read", PolicyCommand.Select, Permissive: false,
        Roles: ["shop_support"], Using: "true", WithCheck: null);

    // Golden value: changing it is a breaking change for existing snapshots
    Assert.Equal(
        """{"Name":"support_read","Command":"Select","Permissive":false"""
        + ""","Roles":["shop_support"],"Using":"true","WithCheck":null}""",
        policy.Serialize());
}
```

Two further tests check tolerant reading in both directions. The snapshot round trip from chapter 23 complements them: the round trip shows that the current code generator and Fluent API agree, the golden value shows that the next version still agrees with the snapshots users already have.

25 Packaging and documentation

Shared settings live in `Directory.Build.props`, package versions centrally in `Directory.Packages.props` (Central Package Management):

Directory.Build.props (shortened)

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  <NoWarn>$(NoWarn);EF1001</NoWarn>
  <Deterministic>true</Deterministic>
</PropertyGroup>
<PropertyGroup>
  <PackageLicenseExpression>MIT</PackageLicenseExpression>
  <PackageReadmeFile>README.md</PackageReadmeFile>
  <VersionPrefix>10.0.0</VersionPrefix>
  <IncludeSymbols>true</IncludeSymbols>
  <SymbolPackageFormat>snupkg</SymbolPackageFormat>
  <PublishRepositoryUrl>true</PublishRepositoryUrl>      <!-- SourceLink -->
  <EmbedUntrackedSources>true</EmbedUntrackedSources>
</PropertyGroup>
```

Element	Why
`GenerateDocumentationFile`	XML docs in the package: users see explanations in the IDE.
`IsAotCompatible`	Analyzers report AOT and trimming problems at build time.
SourceLink, `.snupkg`	Users can debug into the plugin’s code. Built into the SDK since .NET 8.
README in the package	Shown on nuget.org; minimal example and compatibility.
CHANGELOG	Mark breaking changes clearly, especially on EF major upgrades.
`InternalsVisibleTo`	Tests reach internal logic without public API.
Package names	Core without provider, provider packages with a suffix (`.PostgreSQL`, `.SqlServer`).

Besides the API docs, the documentation above all needs the things users do not see in the code: which single services are replaced (composability), which event IDs exist, which limitations apply (pooling) and which security assumptions hold (database roles).

26 The checklist from A to Z

For every new plugin, in the order of development:

Architecture

☐ Decide the level: helpers, model helpers, real plugin or provider?

☐ Separate core and provider packages; types for entity assemblies go into the core.

☐ For every extension point, look for a plugin interface first, replace only then.

Options

☐ Extension immutable with copy constructor and virtual Clone.

☐ All model-relevant options in the hash, nothing that varies per context.

☐ Validate for preconditions (provider, combinations).

☐ Maintain LogFragment and PopulateDebugInfo.

☐ Use method in the namespace Microsoft.EntityFrameworkCore, generic and non-generic.

Services

☐ Registration through the services builder (lifetimes from EF's table).

☐ Decorate single services where the public interface suffices (chapter 5).

☐ Otherwise Replace, and offer the logic publicly and virtually.

☐ ISingletonOptions for singleton services, registered twice.

☐ App dependencies via AddInterceptors, document pooling behavior.

Model

☐ Annotations with a prefix, simple values only, no stored defaults.

☐ Implement the Fluent API non-generically, delegate generically.

☐ Convention builder overloads for ConfigurationSource.

☐ Conventions at ModelFinalizing when they depend on the finished model.

☐ Attributes in the core package, attribute conventions in the provider package.

☐ Validation as IModelFinalizedConvention: throw on errors, warn on questionable cases.

Type mappings

☐ Protected parameter constructor and Clone override.

☐ Literal generation for model and provider type.

☐ Public static Default property, shared converter instance.

☐ JsonValueReaderWriter with Instance and ConstructorExpression.

Queries

☐ EF.Functions methods in the core, translation in the provider package.

☐ Translators: null for foreign methods, correct nullability, set type mappings.

☐ IEvaluatableExpressionFilterPlugin for methods without data reference.

☐ Never insert run-time values at translation time (query cache, AOT).

Run time

☐ Implement interceptors synchronously and asynchronously.

☐ Singleton interceptors stateless and thread-safe.

☐ Own event IDs in an own range, EventDefinitions cached.

☐ Security-relevant cases as warnings, escalatable via ConfigureWarnings.

☐ Where hooks cannot run, offer an explicit API (e.g. ApplyTenantSession).

Migrations & design time

☐ Declarative via annotation provider (design time only) and SQL generator.

☐ Escape identifiers, literals via type mappings, validate self-generated SQL.

☐ Custom operations idempotent, extensions in the migrations namespace.

☐ DesignTimeServicesReference with forProvider; code generator removes annotations.

☐ Check compiled models: Default, Instance, runtime annotations.

Quality

☐ Model, SQL, snapshot round trip and integration tests.

☐ Caching test with different options; test for no phantom migration.

☐ Version range per EF major, CI against previews, EF1001 deliberately.

☐ XML docs, SourceLink, symbols, README, CHANGELOG, limitations documented.

☐ Pin serialized annotation formats with golden-value tests.

☐ Test together with popular plugins (naming conventions, decorators).

Advanced (part VIII)

☐ Document paths that bypass interceptors (ExecuteUpdate, raw SQL).

☐ Configure value generators with HasValueGeneratorFactory(Type) (compiled models).

☐ Check conventions for complex types and JSON columns.

☐ Document and test the order of custom migration operations.

☐ Consider idempotent scripts and suppressTransaction.

☐ Document operational assumptions (roles, pooling, transaction pooling).

☐ Document context lifetimes: factories, background jobs, open connections.

☐ Cross-tenant access only through separate roles, never through app switches.

☐ Measure hot paths (interceptors), do not guess.

☐ CI with preview build and public API analyzer.

Deep extensions (part IX)

☐ Custom SQL nodes: VisitChildren, Update, Quote, Print, Equals/GetHashCode.

☐ Handle custom nodes in the SQL generator AND in the nullability processor.

☐ Aggregates: handle predicates, DISTINCT and empty sets deliberately.

☐ Choose runtime annotations deliberately, filter design-time annotations.

☐ Scaffolding: read the plugin’s schema objects from the database.

☐ Provider-neutral building blocks into the core, differences into provider packages.

Part VIII

Advanced topics

Limits of the run-time hooks, further extension points, operating in production and the way from the plugin into the application.

- Chapter 27: Bulk operations and the limits of interceptors
- Chapter 28: Value generation
- Chapter 29: The update pipeline
- Chapter 30: Complex types and JSON
- Chapter 31: Models per runtime context
- Chapter 32: Advanced migrations
- Chapter 33: Row Level Security in production
- Chapter 34: Plugin performance
- Chapter 35: Using the plugin in ASP.NET Core
- Chapter 36: CI and API stability

27 Bulk operations and the limits of interceptors

A SaveChanges interceptor only sees what goes through `SaveChanges`. `ExecuteUpdate` and `ExecuteDelete` write directly to the database, without change tracker and without SaveChanges. The same applies to raw SQL. A plugin that enforces its guarantees only through SaveChanges has a gap here:

Path	SaveChanges interceptor	Connection interceptor	RLS
`SaveChanges`	yes	yes	yes
`ExecuteUpdate`, `ExecuteDelete`	**no**	yes	yes
`FromSql`, `SqlQuery`, `ExecuteSql`	**no**	yes	yes
ADO.NET or psql as the app role	**no**	**no**	yes (without tenant: no rows)

In the sample, RLS catches every one of these paths because the database enforces the rules itself. An integration test proves it by deliberately bypassing the interceptor:

`tests/...IntegrationTests/AdvancedTests.cs (excerpt)`

```
[Fact]
public async Task ExecuteUpdate_bypasses_interceptor_but_not_rls()
{
    // ... create an order for "tenant" ...
    await using var attacker = ContextFor(tenant);

    // No SaveChanges -> the SaveChanges interceptor does NOT run.
    // The policy's WITH CHECK still prevents the reassignment.
    var ex = await Assert.ThrowsAsync<PostgresException>(() =>
        attacker.Orders.ExecuteUpdateAsync(
            s => s.SetProperty(o => o.TenantId, TenantId.New())));

    Assert.Equal(PostgresErrorCodes.InsufficientPrivilege, ex.SqlState);
}
```

What a plugin must document

Plugins without database-level protection (soft delete, audit fields, tenant filtering via query filter only) must document that bulk operations and raw SQL bypass their rules. To prevent it, an `IQueryExpressionInterceptor` can be used: `ExecuteUpdate` and `ExecuteDelete` go through the query pipeline, the interceptor sees their expression tree and can, for example, reject a `SetProperty` on protected columns. Raw SQL remains open regardless.

28 Value generation

EF generates key values either in the database (identity, sequences) or client-side via a `ValueGenerator`. For `Guid` keys EF does this by itself, for custom types like `TenantId` it does not. A plugin closes this gap with a generator, a factory and a convention:

ValueGeneration/TenantIdValueGenerator.cs (core package)

```
public class TenantIdValueGenerator : ValueGenerator<TenantId>
{
    // Final values, no temporary placeholders until saving
    public override bool GeneratesTemporaryValues => false;

    public override TenantId Next(EntityEntry entry) => TenantId.New();   // UUIDv7
}
```

ValueGeneration/TenantIdValueGeneratorFactory.cs (core package)

```
public class TenantIdValueGeneratorFactory : ValueGeneratorFactory
{
    // Stateless generator: EF may cache it and share it between contexts
    public override ValueGenerator Create(IProperty property, ITypeBase typeBase)
        => new TenantIdValueGenerator();
}
```

Conventions/TenantIdKeyValueGenerationConvention.cs (core package)

```
public virtual void ProcessModelFinalizing(
    IConventionModelBuilder modelBuilder,
    IConventionContext<IConventionModelBuilder> context)
{
    foreach (var entityType in modelBuilder.Metadata.GetEntityTypes())
    {
        if (entityType.BaseType is not null
            || entityType.FindPrimaryKey() is not { Properties: [var property] }
            || property.ClrType != typeof(TenantId))
            continue;

        // Convention source: an explicit ValueGeneratedNever() wins (-> null)
        if (property.Builder.ValueGenerated(ValueGenerated.OnAdd) is null)
            continue;

        property.Builder.HasValueGeneratorFactory(typeof(TenantIdValueGeneratorFactory));
    }
}
```

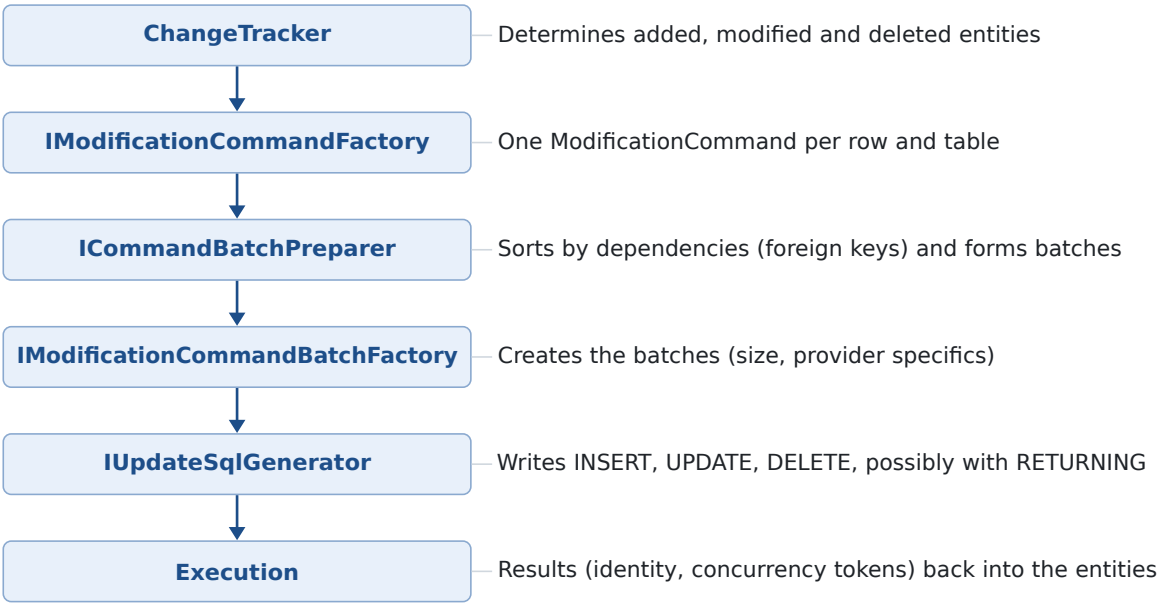
A factory type, not a generator

A value generator can be configured in several ways: `HasValueGenerator` with a type or a factory function, or `HasValueGeneratorFactory` with a factory type. EF Core 10's compiled model generator only accepts the last one and otherwise aborts with "The property ... has a value generator configured. Use 'HasValueGeneratorFactory' to configure the value generator factory type." Plugins should therefore use `HasValueGeneratorFactory(Type)` and give the factory a public parameterless constructor.

The convention lives in the core package because it has nothing to do with PostgreSQL; the provider package plugs it in via its convention set plugin. The compiled model test checks that the factory appears in the generated code.

29 The update pipeline

Chapter 11 describes the way from LINQ to SQL. Writing has its own pipeline, which plugins extend for bulk operations, database-side soft deletes or returned values:



Service	Lifetime	Kind
`IModificationCommandFactory`	Singleton	single
`ICommandBatchPreparer`	Scoped	single
`IModificationCommandBatchFactory`	Scoped	single
`IUpdateSqlGenerator`	Singleton	single

All of these are single provider services, with the consequences from chapter 5. Before replacing them, it pays to check the alternatives: a SaveChanges interceptor can rewrite states (delete becomes modify for soft delete), a command interceptor can add SQL, and for bulk changes there is `ExecuteUpdate`. A custom `IUpdateSqlGenerator` is only justified when the SQL itself needs a different shape, for example `INSERT ... ON CONFLICT` for upserts.

30 Complex types and JSON

Complex types

Complex types (since EF Core 8) are value types without their own identity whose properties end up in the owner's table. For plugins this means: conventions iterating `entityType.GetProperties()` do **not** see their properties. They hang off `IComplexProperty` and must be iterated separately:

Pattern for conventions (sketch)

```
foreach (var complexProperty in entityType.GetComplexProperties())
{
    foreach (var property in complexProperty.ComplexType.GetProperties())
    {
        // Column names are prefixed (e.g. "Address_City"); always determine them via
        // property.GetColumnName(table), never assemble them yourself
    }
}
```

Fluent API methods for properties should additionally be offered on `ComplexTypePropertyBuilder` where they make sense there. The sample plugin does not need this: isolation and value generation only concern entities.

JSON columns

Owned types and (since EF Core 10) complex types can be stored as a single JSON column with `ToJson()`. Values in it are not read and written through converters but through the type mapping's `JsonValueReaderWriter` (chapter 10). Without it, model building fails. The sample's unit and integration tests check both sides:

tests/.../JsonTests.cs

```
using var ctx = Contexts.Create(model: m =>
    m.Entity<Shipment>().OwnsOne(s => s.Details, d => d.ToJson()));

var property = ctx.DesignModel()
    .FindEntityType(typeof(ShipmentDetails))!
    .FindProperty(nameof(ShipmentDetails.ApprovedBy))!;

Assert.Same(TenantIdJsonReaderWriter.Instance,
    property.GetTypeMapping().JsonValueReaderWriter);
```

> **Note**
>
> PostgreSQL arrays (`List<TenantId>` as `uuid[]`) are not JSON: Npgsql maps primitive collections natively as arrays. Whether a provider supports collections with converted element types is provider-specific and deserves its own test before you document it.

31 Models per runtime context

EF builds the model once per context type and caches it (chapter 2). If the model should depend on run-time data, such as one schema per tenant, the cache key must contain that data. That is what `IModelCacheKeyFactory` is for:

tests/.../SchemaPerTenantTests.cs (shortened)

```
public sealed class SchemaModelCacheKeyFactory : IModelCacheKeyFactory
{
    public object Create(DbContext context, bool designTime)
        => context is ISchemaScoped scoped
            ? (context.GetType(), scoped.Schema, designTime)
            : (object)(context.GetType(), designTime);
}

public sealed class SchemaContext : DbContext, ISchemaScoped
{
    // Constructor: (DbContextOptions<SchemaContext> options, string schema)
    public string Schema { get; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
        => modelBuilder.HasDefaultSchema(Schema);
}

// Registration
options.UseNpgsql(connectionString)
       .ReplaceService<IModelCacheKeyFactory, SchemaModelCacheKeyFactory>();
```

The test `SchemaPerTenantTests` checks both sides of the cache key: two schemas produce two models with their own default schema, and the same schema reuses the cached model instead of building it again.

This is the basis for “schema per tenant”, an alternative to RLS. The three usual approaches compared:

Approach	Strengths	Weaknesses
Shared tables + RLS (sample)	One model, one schema, simple migrations; the database enforces separation	All tenants share tables and indexes; misconfigured roles undermine separation
Schema per tenant	Strong separation, separate backups per schema possible	One model per tenant in the cache (memory!), migrations for every schema
Database per tenant	Maximum separation, independent scaling	One model suffices, but connections, migrations and operations per database

> **Model cache and memory**
>
> Every key value produces its own model in memory. With hundreds of tenants and large models this becomes expensive. In addition, values that change the model must never come from unchecked input.

32 Advanced migrations

Order of operations

Generated and hand-written operations run in the order they appear in `Up()`. This is a real source of errors. In the sample, the policy `support_read` applies `TO shop_support`. EF generates it directly after `CREATE TABLE`. If the role is only created at the end of `Up()`, the migration fails.

Migrations/20261021_Initial.cs

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // 1. Whatever policies reference: BEFORE the generated operations
    migrationBuilder.CreateApplicationRole("shop_support");

    // 2. Generated by EF: tables, RLS, policies (TO shop_support)
    migrationBuilder.CreateTable(name: "Customers", /* ... */);

    // 3. Whatever requires the tables: AFTER the generated operations
    migrationBuilder.CreateApplicationRole("shop_app", canLogin: true);
    migrationBuilder.GrantTablePrivileges("shop_app");     // applies to existing tables
}
```

Idempotent scripts

`dotnet ef migrations script --idempotent` produces a script that can run against any database state. For this, Npgsql wraps every operation in a `DO $EF$ ... $EF$` block that checks the history table. Custom operations that produce a `DO $$` block themselves end up inside it. This works because the dollar quotes use different tags. Assumptions like this belong in a test:

tests/...IntegrationTests/AdvancedTests.cs (excerpt)

```
var script = db.GetService<IMigrator>()
    .GenerateScript(options: MigrationsSqlGenerationOptions.Idempotent);

await db.Database.ExecuteSqlRawAsync(script);   // nested DO valid?
await db.Database.ExecuteSqlRawAsync(script);   // second run is skipped
```

Operations outside of transactions

Some commands must not run inside a transaction, in PostgreSQL for example `CREATE INDEX CONCURRENTLY`. Custom operations with such SQL end their command with `EndStatement(builder, suppressTransaction: true)`. EF then executes it outside the migration transaction. The price: if the migration fails afterwards, this command is not rolled back; it should therefore be idempotent.

Migration bundles

`dotnet ef migrations bundle` produces an executable that applies migrations without the SDK. It contains the startup project including the plugin assemblies; custom operations and the plugin's SQL generator therefore take effect there too. It is important that the design-time factory or host uses the same plugin options as the application, otherwise SQL for a different model is produced.

33 Row Level Security in production

RLS is only as strong as its operating conditions. These pitfalls silently render the policies ineffective or unusable:

Pitfall	Consequence	Remedy
App runs as table owner	Policies are ignored	`FORCE ROW LEVEL SECURITY` (plugin default) and separate roles
Superuser or `BYPASSRLS`	RLS is always bypassed, even with FORCE	App role with `NOSUPERUSER NOBYPASSRLS` (custom operation, chapter 19)
Setting not set	Without `nullif`: error; with it: empty results	The interceptor sets the value on every open
No index on the tenant column	Every query scans the whole table	Index convention (chapter 8)
Transaction pooling (PgBouncer)	Session settings reach other clients or get lost	Set the setting transaction-locally (see below)
Connection opened outside EF	The session keeps an old tenant or none	Let EF open connections, or `ApplyTenantSession()` (chapter 15)
“Bypass” switch for the app role	Anyone who can run SQL as the app role can flip it	Separate roles for cross-tenant access (see below)

Transaction pooling

PgBouncer in “transaction” mode shares a server connection between clients per transaction. A session value set with `set_config(..., false)` can then reach another client, a security hole. The safe variant: set the setting *transaction-locally* with `set_config(..., true)` and run all database work in an explicit transaction. Technically, the interceptor moves from `IDbConnectionInterceptor` to `IDbTransactionInterceptor` (`TransactionStarted`). Because queries without a transaction then have no tenant at all, the application must adhere to this consistently. The sample stays with the simpler session model and documents that it requires session or Npgsql pooling.

Policy performance

PostgreSQL inserts policies into every query like additional `WHERE` conditions. `current_setting` is `STABLE` and evaluated once per query; together with the index the overhead is small. Expensive policies come from subqueries (for example permissions from another table) or functions that are not `LEAKPROOF` and prevent optimizations. You can check this with `EXPLAIN (ANALYZE)` as the app role, not as owner, because the owner may not even see the policies.

Legitimate cross-tenant access

Sooner or later someone needs data across tenants: reports for the operator, maintenance jobs, support. The tempting shortcut is a switch in the application, for example an additional session setting `app.bypass` that the policy honors. That is not a security boundary: whatever the application role can set, an injected SQL statement can set too. Separation must come from database roles instead:

Need	Solution	Why it is safe
Reports, analytics	Separate read-only role with `BYPASSRLS`, only `SELECT`, its own connection string and a context configuration without tenant	The app role cannot become this role; its credentials live elsewhere
Support reads selected data	Additional permissive policy for a support role: `HasPolicy(...)` with `For(Select)` and `To("shop_support")`	Restricted to SELECT and to members of the role
Maintenance per tenant	Background job that iterates the tenants and sets each one (chapter 35)	Runs through the normal policies

Reporting role (tests/...IntegrationTests/CrossTenantAccessTests.cs)

```
-- Created by an administrator: BYPASSRLS requires a superuser
CREATE ROLE shop_reporting LOGIN PASSWORD '...' NOSUPERUSER BYPASSRLS;
GRANT USAGE ON SCHEMA public TO shop_reporting;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO shop_reporting;
```

The application role, in contrast, cannot switch the policies off. `SET row_security = off` does not disable RLS for a role without `BYPASSRLS`; PostgreSQL instead rejects every query the policies would affect. `CrossTenantAccessTests` checks all three: the reporting role sees all tenants, it cannot write, and the app role cannot help itself.

> **Audit cross-tenant access**
>
> Every connection of the reporting role is an access to all tenants. Log it (`log_connections`, pgaudit) and grant the role to as few people and services as possible. The plugin's `CreateApplicationRole` deliberately always creates roles with `NOBYPASSRLS`: bypass roles do not belong in the application's migrations.

34 Plugin performance

Plugin code runs at three frequencies. Knowing which code runs how often tells you where optimization pays off:

Frequency	Code	Consequence
Once per provider	Conventions, type mapping plugins, singleton options	Readability over micro-optimization
Once per query shape	Translators, evaluatable filter, query interceptors	Moderate; the result is cached
On every execution	Interceptors for commands, connections, SaveChanges, materialization	Hot path: no reflection, no unnecessary allocations

This leads to a few rules the sample follows: `MethodInfo` and `PropertyInfo` lookups live in static fields. The SaveChanges interceptor only collects entries when there are tenant-relevant changes. The connection interceptor costs one round trip per open, which is documented. Such assumptions should be measured, not guessed. The repository contains a BenchmarkDotNet project for the SaveChanges logic; it runs without a database:

benchmarks/.../SaveChangesInterceptorBenchmark.cs (shortened)

```
[MemoryDiagnoser]
public class SaveChangesInterceptorBenchmark
{
    [Params(1, 100)]
    public int Entities { get; set; }

    // GlobalSetup: context with UseAcmeTenancy() (no connection) and the interceptor

    [Benchmark(Baseline = true)]
    public int TrackOnly()                  // what EF does anyway before saving
    {
        AddOrders();
        var count = _db.ChangeTracker.Entries().Count();
        _db.ChangeTracker.Clear();
        return count;
    }

    [Benchmark]
    public int TrackAndApplyTenant()        // the same plus the plugin's SaveChanges logic
    {
        AddOrders();
        _interceptor.ApplyTenant(_db);
        var count = _db.ChangeTracker.Entries().Count();
        _db.ChangeTracker.Clear();
        return count;
    }
}
```

Running the benchmark

```
dotnet run -c Release --project benchmarks/Acme.EntityFrameworkCore.Tenancy.Benchmarks
```

Method	Entities	Mean	Ratio	Allocated
`TrackOnly`	1	1.5 µs	1.00	1.8 KB
`TrackAndApplyTenant`	1	2.3 µs	1.54	2.7 KB
`TrackOnly`	100	123 µs	1.00	134 KB
`TrackAndApplyTenant`	100	170 µs	1.38	202 KB

Example results (short job, .NET 10, author’s machine; absolute values vary, the ratios are more meaningful). The plugin adds about half a microsecond per saved entity, roughly 40 to 50 percent on top of change tracking alone. That sounds like much, but a single round trip to the database costs hundreds of microseconds; the connection interceptor’s extra round trip per open (chapter 15) therefore weighs far more than the SaveChanges logic. That is the kind of insight a benchmark provides: it shows where optimizing is worthwhile and where it is not.

> **Tip**
>
> For the startup of large models, the compiled model (chapter 21) is the most effective lever. A plugin’s conventions then no longer run at run time at all; that is why it pays to test compiled model support rather than merely claim it.

35 Using the plugin in ASP.NET Core

The sample `samples/ShopApi` shows the way from the plugin into a web API. The tenant comes from the claim `tenant_id` of the signed-in user:

samples/ShopApi/HttpTenantProvider.cs

```
public sealed class HttpTenantProvider : ITenantProvider
{
    public const string ClaimType = "tenant_id";

    private readonly IHttpContextAccessor _httpContextAccessor;

    public HttpTenantProvider(IHttpContextAccessor httpContextAccessor)
    {
        ArgumentNullException.ThrowIfNull(httpContextAccessor);

        _httpContextAccessor = httpContextAccessor;
    }

    // Determined anew on every access: also works as a singleton (pooling)
    public TenantId? CurrentTenantId
    {
        get
        {
            var value = _httpContextAccessor.HttpContext?.User.FindFirst(ClaimType)?.Value;
            return Guid.TryParse(value, out var id) ? new TenantId(id) : null;
        }
    }
}
```

samples/ShopApi/Program.cs (excerpt)

```
builder.Services.AddHttpContextAccessor();
builder.Services.AddAcmeTenancy<HttpTenantProvider>();
builder.Services.AddDbContext<ShopApiContext>((sp, options) => options
    .UseNpgsql(builder.Configuration.GetConnectionString("Shop"))
    .UseAcmeTenancy(sp));

// No tenant filter in code: RLS in the database provides the separation
app.MapGet("/orders", async (ShopApiContext db) =>
    await db.Orders.OrderBy(o => o.Id).ToListAsync());
```

> **Where the tenant comes from**
>
> The tenant must come from a trustworthy source: a claim from a signed token. A header or query parameter that the client sets freely undermines the entire isolation. The sample contains a header simulation for the development environment only and labels it explicitly.

For `AddDbContextPool`, register the provider as a singleton. This is possible because `HttpTenantProvider` determines the tenant anew on every access; the interceptors ask it on every connection and every save:

Variant with pooling

```
builder.Services.AddAcmeTenancy<HttpTenantProvider>(ServiceLifetime.Singleton);
builder.Services.AddDbContextPool<ShopApiContext>((sp, options) => options
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp));   // sp: root provider; interceptors are created once
```

DbContext factories

`AddDbContextFactory` is the usual choice in Blazor Server and for parallel work. Unlike `AddDbContext`, it registers the factory and the options as **singletons** by default. The options callback then receives the root provider, not the scope of the current request. For a scoped tenant provider that holds state per scope, this has consequences:

Registration	Behavior
Singleton factory, scope validation on (ASP.NET Core in Development)	Exception: “Cannot resolve scoped service ’ITenantProvider’ from root provider.”
Singleton factory, scope validation off (production default)	No error. The interceptors get an instance from the root container that no request ever fills: no tenant, saving throws, queries see nothing
Factory registered as `Scoped`	Every scope gets its own factory with its own tenant provider: correct

Factory with the tenant of the request

```
builder.Services.AddDbContextFactory<ShopApiContext>(
    (sp, options) => options.UseNpgsql(connectionString).UseAcmeTenancy(sp),
    ServiceLifetime.Scoped);         // factory and options per scope
```

A provider that determines the tenant anew on every access, such as `HttpTenantProvider`, works in every variant; the problem affects providers with state per scope. `AddPooledDbContextFactory` always behaves like pooling: the options exist once, so the tenant provider must be such a singleton. `DbContextFactoryTests` shows all three variants.

Outside of HTTP requests

Background services, message handlers and scheduled jobs have no request and no claim. The tenant then comes from the job itself, for example from the message or from a list of tenants to process. The sample’s `AmbientTenantProvider` stores it in an `AsyncLocal`, which flows with the asynchronous control flow:

samples/ShopSample/AmbientTenantProvider.cs (shortened)

```
public sealed class AmbientTenantProvider : ITenantProvider
{
    private static readonly AsyncLocal<TenantId?> Current = new();

    public TenantId? CurrentTenantId => Current.Value;

    public static IDisposable Use(TenantId tenant)
    {
        var previous = Current.Value;
        Current.Value = tenant;
        return new Restore(previous);     // sets the previous tenant back on Dispose
```

```
    }

    // private sealed class Restore : IDisposable { ... Current.Value = _previous; }
}
```

Job pattern (tests/.../BackgroundTenantTests.cs)

```
// Tenant first, then scope and context: the interceptors are created inside the scope
using (AmbientTenantProvider.Use(tenant))
{
    await using var scope = scopeFactory.CreateAsyncScope();
    var db = scope.ServiceProvider.GetRequiredService<ShopContext>();
    // ... work for this tenant ...
}
```

Because the provider reads the `AsyncLocal` on every access, it can be registered as a singleton. Parallel jobs keep their own tenant, and after the `using` block no tenant is left behind; `BackgroundTenantTests` checks both with 20 parallel jobs. An application with web requests *and* jobs combines both sources in one provider: the claim first, otherwise the ambient tenant.

> **Never fall back to “all tenants”**
>
> Without a tenant, a job must fail or see nothing, never everything. The plugin behaves this way: the session value stays empty, the policies match no rows and saving throws. Jobs that really need all tenants use the reporting role from chapter 33.

36 CI and API stability

The workflow

The GitHub Actions workflow in the repository implements the recommendations from chapters 23 and 24. One job builds and tests without and with a database, a second builds the handbook:

.github/workflows/ci.yml (shortened)

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { global-json-file: global.json }
      - run: dotnet build -c Release
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.Tests -c Release --no-build
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.IntegrationTests
          -c Release --no-build
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.SqlServer.Tests
          -c Release --no-build

  handbook:                           # both PDFs with LuaLaTeX, in docs/handbook
    runs-on: ubuntu-latest
    container: texlive/texlive@sha256:540d…   # TeX Live 2026, pinned by digest
    steps:
      # ... checkout ...
      - run: python3 check_code.py     # fails on overlong code lines
      - run: latexmk EF-Core-Plugin-Handbook.tex
```

The sample deliberately builds against stable versions only (.NET 10, EF Core 10). An additional build against the next EF prerelease, as recommended in chapter 24, reveals breaks in internal APIs earlier. It also needs the matching .NET prerelease SDK, though, because every EF Core major version only ships for the .NET version of the same number (EF Core 11 only for `net11.0`). If you set one up, run it as a separate job with `continue-on-error` so that it does not block the regular CI.

Protecting the public API

A plugin has two public APIs: the one for users (Fluent API, options, types) and the one for extenders (virtual methods, dependencies, public services). Both break easily and unnoticed. The `Microsoft.CodeAnalysis.PublicApiAnalyzers` package keeps the public API in two text files (`PublicAPI.Shipped.txt`, `PublicAPI.Unshipped.txt`) and reports every deviation. Every API change thus becomes visible as a diff in the pull request.

> **Setup**
>
> On the first build, the analyzer reports every public API as RS0016 because the files are empty. The code fix “Add all items ... to the public API” fills them. At release time, the content moves from Unshipped to Shipped; from then on, removing a shipped API is a visible breaking change.

Part IX

Deep extensions

Custom SQL expressions, aggregates, run-time metadata, scaffolding and a second provider: the tools for very complex plugins.

- Chapter 37: Custom SQL expression nodes
- Chapter 38: Aggregate translators
- Chapter 39: Property-level Fluent API and runtime annotations
- Chapter 40: Scaffolding in practice
- Chapter 41: What belongs in the core
- Chapter 42: A second provider: SQL Server

37 Custom SQL expression nodes

Chapter 12 builds SQL from existing building blocks: functions, constants, casts. That is enough for most plugins. A **custom expression node** (a subclass of `SqlExpression`) becomes necessary when at least one of the following applies:

Reason	Example
The syntax is not a function call	Operators such as `@@`, `<->`, `IS DISTINCT FROM`, keywords such as `AT TIME ZONE`
The nullability semantics differ	`IS DISTINCT FROM` is never NULL, even when an operand is NULL
Other parts should recognize the node	Optimizations that only apply to this expression

The sample implements the null-safe comparison `EF.Functions.IsNotDistinctFrom(a, b)` and `IsDistinctFrom(a, b)`. Npgsql does not offer it as a function; it meets all three reasons.

Anatomy of a node

Member	Purpose	Consequence if missing
`VisitChildren` / `Update`	Visitors can replace children; unchanged children yield the same instance	EF throws at the first visitor
`Equals` / `GetHashCode`	Structural comparison for the query cache and optimizations	Cache misses, wrong deduplication
`Print`	Readable representation in DebugView and error messages	Unclear errors
`Quote`	C# code for precompiled queries and NativeAOT	No AOT support
SQL generator	Writes the node as SQL	**Silently** missing SQL
Nullability processor	Knows the NULL semantics	Exception during translation

Query/Expressions/DistinctFromExpression.cs (shortened)

```
public class DistinctFromExpression : SqlExpression
{
    private static ConstructorInfo? _quotingConstructor;

    public DistinctFromExpression(SqlExpression left, SqlExpression right, bool negated,
        RelationalTypeMapping? typeMapping)
        : base(typeof(bool), typeMapping)
    {
        // ... ThrowIfNull ...
        Left = left;
        Right = right;
        IsNegated = negated;
    }

    public virtual SqlExpression Left { get; }
    public virtual SqlExpression Right { get; }
```

```
    public virtual bool IsNegated { get; }

    protected override Expression VisitChildren(ExpressionVisitor visitor)
        => Update((SqlExpression)visitor.Visit(Left), (SqlExpression)visitor.Visit(Right));

    public virtual DistinctFromExpression Update(SqlExpression left, SqlExpression right)
        => left == Left && right == Right
            ? this
            : new DistinctFromExpression(left, right, IsNegated, TypeMapping);

    public override Expression Quote()
        => New(
            _quotingConstructor ??= typeof(DistinctFromExpression).GetConstructor(
                [typeof(SqlExpression), typeof(SqlExpression), typeof(bool),
                 typeof(RelationalTypeMapping)])!,
            Left.Quote(), Right.Quote(), Constant(IsNegated),
            RelationalExpressionQuotingUtilities.QuoteTypeMapping(TypeMapping));

    protected override void Print(ExpressionPrinter expressionPrinter)
    {
        expressionPrinter.Visit(Left);
        expressionPrinter.Append(IsNegated ? " IS DISTINCT FROM " : " IS NOT DISTINCT FROM ");
        expressionPrinter.Visit(Right);
    }

    public override bool Equals(object? obj)
        => obj is DistinctFromExpression other
           && (ReferenceEquals(this, other)
               || base.Equals(other) && Left.Equals(other.Left)
                  && Right.Equals(other.Right) && IsNegated == other.IsNegated);

    public override int GetHashCode()
        => HashCode.Combine(base.GetHashCode(), Left, Right, IsNegated);
}
```

The translator creates the node and aligns the type mappings of both sides: if one side has a mapping (a column, say), the other gets the same one. That way a constant becomes the matching literal:

Query/DistinctFromTranslator.cs (excerpt)

```
var definition = method.GetGenericMethodDefinition();
if (definition != IsNotDistinctFromMethod && definition != IsDistinctFromMethod)
    return null;

var left = arguments[1];                        // arguments[0] is DbFunctions
var right = arguments[2];
var typeMapping = ExpressionExtensions.InferTypeMapping(left, right)
                  ?? _typeMappingSource.FindMapping(left.Type);

return new DistinctFromExpression(
    _sql.ApplyTypeMapping(left, typeMapping)!,
    _sql.ApplyTypeMapping(right, typeMapping)!,
    negated: definition == IsDistinctFromMethod,
    _typeMappingSource.FindMapping(typeof(bool)));
```

Extending the SQL generator

The `QuerySqlGenerator` writes SQL using a visitor. It hands unknown nodes to `ExpressionVisitor.VisitExtension`, which only visits the children. Without an override there is no error, but **SQL without the operator**. The plugin derives from the Npgsql generator:

Query/Internal/TenancyNpgsqlQuerySqlGenerator.cs (shortened)

```
public class TenancyNpgsqlQuerySqlGenerator : NpgsqlQuerySqlGenerator
{
    public TenancyNpgsqlQuerySqlGenerator(QuerySqlGeneratorDependencies dependencies,
        IRelationalTypeMappingSource typeMappingSource,
        bool reverseNullOrderingEnabled, Version postgresVersion)
        : base(dependencies, typeMappingSource, reverseNullOrderingEnabled, postgresVersion)
    {
    }

    protected override Expression VisitExtension(Expression extensionExpression)
        => extensionExpression switch
        {
            DistinctFromExpression e => VisitDistinctFrom(e),
            _ => base.VisitExtension(extensionExpression),
        };

    protected virtual Expression VisitDistinctFrom(DistinctFromExpression expression)
    {
        VisitOperand(expression.Left);
        Sql.Append(expression.IsNegated ? " IS DISTINCT FROM " : " IS NOT DISTINCT FROM ");
        VisitOperand(expression.Right);
        return expression;
    }
}
```

The generator is created through `IQuerySqlGeneratorFactory`, a singleton single service. The plugin replaces it with its own factory taking the same dependencies as Npgsql's (`QuerySqlGeneratorDependencies`, type mapping source, `INpgsqlSingletonOptions`).

Extending the nullability processor

After translation, EF optimizes NULL comparisons. To do so it must know for every node whether it can become NULL. Unknown nodes end up in `VisitCustomSqlExpression`, whose base implementation throws an exception. The processor hangs off a chain of three classes, all of which must be replaced:

Query/Internal/TenancyNpgsqlSqlNullabilityProcessor.cs (shortened)

```
public class TenancyNpgsqlSqlNullabilityProcessor : NpgsqlSqlNullabilityProcessor
{
    // Constructor: (…ProcessorDependencies, …ProcessorParameters) : base(...)

    protected override SqlExpression VisitCustomSqlExpression(
        SqlExpression sqlExpression, bool allowOptimizedExpansion, out bool nullable)
    {
        if (sqlExpression is DistinctFromExpression distinctFrom)
        {
            // Visit children (they can be optimized), their nullability does not matter
```

```
            var left = Visit(distinctFrom.Left, out _)!;
            var right = Visit(distinctFrom.Right, out _)!;

            nullable = false;                   // IS [NOT] DISTINCT FROM is never NULL
            return distinctFrom.Update(left, right);
        }

        return base.VisitCustomSqlExpression(
            sqlExpression, allowOptimizedExpansion, out nullable);
    }
}

public class TenancyNpgsqlParameterBasedSqlProcessor : NpgsqlParameterBasedSqlProcessor
{
    protected override Expression ProcessSqlNullability(
        Expression selectExpression, ParametersCacheDecorator parametersDecorator)
        => new TenancyNpgsqlSqlNullabilityProcessor(Dependencies, Parameters)
            .Process(selectExpression, parametersDecorator);
}

// + TenancyNpgsqlParameterBasedSqlProcessorFactory
//   : IRelationalParameterBasedSqlProcessorFactory
```

Infrastructure/TenancyOptionsExtension.cs › ApplyServices (addition)

```
// 4) Deep extension: custom SQL expression node
services.Replace(ServiceDescriptor
    .Singleton<IQuerySqlGeneratorFactory, TenancyNpgsqlQuerySqlGeneratorFactory>());
services.Replace(ServiceDescriptor
    .Scoped<IRelationalParameterBasedSqlProcessorFactory,
        TenancyNpgsqlParameterBasedSqlProcessorFactory>());
```

The price of custom nodes

Custom nodes require replacing two single services and deriving from internal Npgsql classes. Two plugins that both introduce custom nodes can only be combined if one derives from the other. Hence: function composition first; custom nodes only when one of the three reasons applies. And the replacement belongs prominently in the documentation.

Experimental API

`Quote` and `RelationalExpressionQuotingUtilities` belong to the experimental API for precompiled queries. The compiler reports their use as EF9100; the sample suppresses it project-wide and documents this in `Directory.Build.props`.

tests/.../DeepQueryTests.cs (excerpt)

```
[Fact]
public void IsDistinctFrom_in_projection_passes_nullability_processor()
{
    // Without a custom nullability processor, EF would throw here
    using var ctx = Contexts.Create();

    var sql = ctx.Customers
        .Select(c => EF.Functions.IsDistinctFrom(c.Name, (string?)null))
        .ToQueryString();
```

```
    Assert.Contains("IS DISTINCT FROM", sql);
}
```

38 Aggregate translators

Aggregates such as `Sum` or `string_agg` combine several rows, typically after `GroupBy`. An `IAggregateMethodCallTranslator` receives, instead of a single argument, an `Enumerable Expression` that describes *what* is being aggregated:

Property	Meaning	Origin in LINQ
`Selector`	The aggregated expression	`g.Select(o => o.Total > 0)`
`IsDistinct`	Distinct values only	`g.Select(...).Distinct()`
`Predicate`	Filter before the aggregate	`g.Where(o => o.Id > 10)`
`Orderings`	Order (for order-dependent aggregates)	`g.OrderBy(...)`

The sample adds `EF.Functions.Every(values)` for PostgreSQL's `bool_and`:

`Query/EveryAggregateTranslator.cs (excerpt)`

```
public virtual SqlExpression? Translate(MethodInfo method, EnumerableExpression source,
    IReadOnlyList<SqlExpression> arguments,
    IDiagnosticsLogger<DbLoggerCategory.Query> logger)
{
    if (method != EveryMethod || source.Selector is not SqlExpression selector)
        return null;

    // Npgsql's factory implements DISTINCT, FILTER (WHERE ...) and ORDER BY in aggregates
    if (_sql is not NpgsqlSqlExpressionFactory npgsql)
        return null;

    var boolean = _typeMappingSource.FindMapping(typeof(bool))!;

    // bool_and returns NULL for an empty set, Enumerable.All returns true
    return _sql.Coalesce(
        npgsql.AggregateFunction("bool_and", [selector], source,
            nullable: true, argumentsPropagateNullability: [false],
            typeof(bool), boolean),
        _sql.Constant(true, boolean),
        boolean);
}
```

`Generated SQL for g.Where(o => o.Id > 10).Select(o => o.Total > 0)`

```
SELECT COALESCE(bool_and(o."Total" > 0.0) FILTER (WHERE o."Id" > 10), TRUE)
FROM "Orders" AS o
GROUP BY o."TenantId"
```

> **Semantics before syntax**
>
> An aggregate translator must match the semantics of the .NET side, not just the database syntax. Typical pitfalls: empty sets (NULL instead of 0, true or an empty string), NULL values in the input, and predicates. Npgsql itself aligns `string.Join` with `string_agg` using `COALESCE`; the sample follows this pattern.

Databases without a `FILTER` clause (such as SQL Server) emulate predicates with `CASE WHEN predicate THEN value END`. A provider-neutral aggregate translator must know both; that is why this one lives in the PostgreSQL package.

39 Property-level Fluent API and runtime annotations

Property builder

So far the plugin assumes that the tenant property is called `TenantId` and that `ITenantOwned` is implemented. With a Fluent API method on `PropertyBuilder` it works without either, for example with existing models using a property `MandantId` or an explicit interface implementation:

Usage

```
modelBuilder.Entity<Contract>().Property(c => c.Mandant).IsTenantProperty();
```

Metadata/TenancyPropertyBuilderExtensions.cs (core package)

```
public static PropertyBuilder IsTenantProperty(
    this PropertyBuilder builder, bool isTenant = true)
{
    ArgumentNullException.ThrowIfNull(builder);

    if (isTenant)
        builder.Metadata.SetAnnotation(TenancyAnnotationNames.IsTenantProperty, true);
    else
        builder.Metadata.RemoveAnnotation(TenancyAnnotationNames.IsTenantProperty);

    return builder;
}

public static PropertyBuilder<TProperty> IsTenantProperty<TProperty>(
    this PropertyBuilder<TProperty> builder, bool isTenant = true)
    => (PropertyBuilder<TProperty>)IsTenantProperty((PropertyBuilder)builder, isTenant);

public static IConventionPropertyBuilder? IsTenantProperty(
    this IConventionPropertyBuilder builder,
    bool isTenant = true,
    bool fromDataAnnotation = false)
    => builder.HasAnnotation(TenancyAnnotationNames.IsTenantProperty,
           isTenant ? true : null, fromDataAnnotation) is null
        ? null
        : builder;
```

The method lives in the core package because it only sets metadata. The conventions of both provider packages find the tenant property through the same core function:

Metadata/TenancyEntityTypeExtensions.cs (core package, shortened)

```
public static IReadOnlyProperty? FindTenantPropertyCandidate(
    this IReadOnlyEntityType entityType)
{
    var marked = entityType.GetProperties()
        .Where(p => p[TenancyAnnotationNames.IsTenantProperty] is true)
        .ToList();

    if (marked.Count > 1)
```

```
            throw new InvalidOperationException("... several tenant properties ...");
        if (marked.Count == 1)
            return marked[0];

        return typeof(ITenantOwned).IsAssignableFrom(entityType.ClrType)
            ? entityType.FindProperty(nameof(ITenantOwned.TenantId))
            : null;
    }
```

Builder	Typical plugin methods	Note
`PropertyBuilder` / `<T>`	Column options, roles of properties	Here: `IsTenantProperty`
`IndexBuilder` / `<T>`	Index methods, operator classes, filters	Annotation on `Metadata` (IMutableIndex)
`KeyBuilder`	Key options (clustering)	As above
`ComplexTypePropertyBuilder`	Properties in complex types	Only offer where the option makes sense there (chapter 30)
`ModelBuilder`	Model-wide options	Annotation on the model

Runtime annotations

The SaveChanges interceptor needs to know at run time which entities are isolated and what their tenant property is called. The convention writes this into an annotation on the entity type; the interceptor reads it from the *runtime model*:

Convention and core package (excerpt)

```
// Convention (design-time model)
entityType.Builder.HasAnnotation(TenancyAnnotationNames.TenantProperty, property.Name);

// Interceptor (runtime model)
public static string? GetTenantPropertyName(this IReadOnlyEntityType entityType)
    => entityType.GetRootType()[TenancyAnnotationNames.TenantProperty] as string;
```

Two questions decide whether this works. Does the annotation reach the runtime model? Yes: `RuntimeModelConvention` only removes EF's own design-time annotations and keeps all others. Does it reach the compiled model? Yes: EF writes simple values automatically (chapter 21). The custom `ICSharpRuntimeAnnotationCodeGenerator` therefore has the opposite job: it removes the RLS annotations that only migrations need:

Design/Internal/TenancyCSharpRuntimeAnnotationCodeGenerator.cs (shortened)

```
public class TenancyCSharpRuntimeAnnotationCodeGenerator
    : NpgsqlCSharpRuntimeAnnotationCodeGenerator
{
    // Constructor: (CSharpRuntimeAnnotationCodeGeneratorDependencies,
    //               RelationalCSharpRuntimeAnnotationCodeGeneratorDependencies) : base(...)

    public override void Generate(IEntityType entityType,
        CSharpRuntimeAnnotationCodeGeneratorParameters parameters)
    {
        if (!parameters.IsRuntime)
        {
            foreach (var key in parameters.Annotations.Keys
```

```
                            .Where(k => k.StartsWith(RlsAnnotationNames.Prefix,
                                                     StringComparison.Ordinal))
                            .ToList())
                {
                    parameters.Annotations.Remove(key);
                }
            }

            base.Generate(entityType, parameters);
        }
    }
```

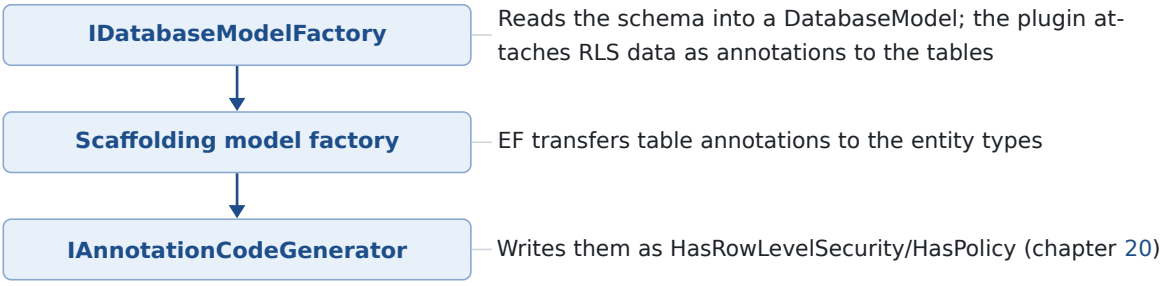
When the compiled model is generated for NativeAOT (`ForNativeAot`), EF also writes the relational model. The annotation provider has copied the RLS annotations onto the tables, so they appear a second time there. The generator therefore applies the same filter in an override of `Generate(ITable, ...)`.

Annotation	Design time	Run time	Compiled model
`Acme:Rls:*` (policies)	yes (migrations)	yes, unused	filtered
`Acme:Tenancy:TenantProperty`	yes	yes (interceptor)	yes
`Acme:Tenancy:IsTenantProperty`	yes (convention)	yes, unused	yes (small)

> **Tip**
>
> For every annotation it should be clear who reads it and when. Annotations read only at design time do not belong in the compiled model; those read at run time must be in it and need a test against the generated model. The sample's compiled model test checks both.

40 Scaffolding in practice

When scaffolding (`dotnet ef dbcontext scaffold`), EF reads the schema and generates entities and a context. A plugin managing its own schema objects should recognize them in the process. The route has three stations, two of which the plugin already knows:



Scaffolding/TenancyDatabaseModelFactory.cs (shortened)

```
public class TenancyDatabaseModelFactory : NpgsqlDatabaseModelFactory
{
    public TenancyDatabaseModelFactory(
        IDiagnosticsLogger<DbLoggerCategory.Scaffolding> logger)
        : base(logger)
    {
    }

    public override DatabaseModel Create(
        DbConnection dbConnection, DatabaseModelFactoryOptions options)
    {
        var model = base.Create(dbConnection, options);
        // ... open the connection if needed ...
        ReadRowLevelSecurity(dbConnection, model);    // pg_class.relrowsecurity / ...force...
        ReadPolicies(dbConnection, model);            // pg_policies
        return model;
    }

    protected virtual void ReadPolicies(DbConnection connection, DatabaseModel model)
    {
        // SELECT schemaname, tablename, policyname, permissive, roles::text[], cmd,
        //        qual, with_check FROM pg_catalog.pg_policies
        // ...
        var policy = new RlsPolicy(
            Name: reader.GetString(2),
            Command: Enum.Parse<PolicyCommand>(reader.GetString(5), ignoreCase: true),
            Permissive: reader.GetString(3) == "PERMISSIVE",
            Roles: roles is ["public"] ? null : roles,    // "public" = default without TO
            Using: reader.IsDBNull(6) ? null : reader.GetString(6),
            WithCheck: reader.IsDBNull(7) ? null : reader.GetString(7));

        table[RlsAnnotationNames.PolicyPrefix + policy.Name] = policy.Serialize();
    }
}
```

The factory is registered like the code generator in the design-time services, with `RemoveAll` and `AddSingleton`. The data format is the same as with the Fluent API; snapshot, migrations and code generator therefore work without further changes.

> **Normalized expressions**
>
> PostgreSQL stores policy expressions in normalized form, for example `(("TenantId" = (NULLIF( current_setting('app.tenant_id'::text, true), ''::text))::uuid))`. Scaffolded policies contain exactly this text. That is correct, but looks different from the form generated by the convention. Teams switching to migrations after scaffolding should unify the policies once, otherwise the first migration produces a DROP/CREATE with identical content.

tests/...IntegrationTests/AdvancedTests.cs (excerpt)

```
[Fact]
public void Scaffolding_reads_rls_and_policies()
{
    using var ctx = ContextFor(null);
    var factory = new TenancyDatabaseModelFactory(
        ctx.GetService<IDiagnosticsLogger<DbLoggerCategory.Scaffolding>>());

    var model = factory.Create(
        _fixture.OwnerConnectionString, new DatabaseModelFactoryOptions());
    var orders = model.Tables.Single(t => t.Name == "Orders");

    Assert.Equal(true, orders[RlsAnnotationNames.Enabled]);
    Assert.Equal(true, orders[RlsAnnotationNames.Forced]);
}
```

41 What belongs in the core

A second provider is the acid test for the package architecture. It shows which parts are truly provider-independent. The rule of thumb: **whatever knows no SQL syntax and no provider type belongs in the core.** For the sample this yields:

Building block	Location	Reason
`TenantId`, converter, JSON reader/writer	Core	Pure .NET types
`ITenantOwned`, `ITenantProvider`, attributes	Core	For entity assemblies without a provider dependency
`IsTenantProperty()`, candidate lookup	Core	Metadata only
Runtime annotation, `GetTenantPropertyName`	Core	Read by the provider-neutral interceptor
SaveChanges interceptor	Core	Works only with the change tracker
`TenantSessionInterceptor` (abstract)	Core	Same flow, only the SQL command differs
Evaluatable filter with attribute	Core	Recognizes function classes of all packages
Value generation, logging, `AddAcmeTenancy`	Core	No SQL involved
`Database.ApplyTenantSession()`	Core	Uses the provider's interceptor
Type mapping, translators, migrations, RLS conventions	Provider	Store types, SQL syntax, schema objects
Custom SQL nodes, aggregates	Provider	Depend on syntax and provider factory

> **Name collisions between provider packages**
>
> Two provider packages that both define an extension method `UseAcmeTenancy` on `DbContextOptionsBuilder` cannot be referenced at the same time: the call would be ambiguous. The PostgreSQL package uses EF's builder, the SQL Server package the provider's builder (`UseSqlServer(cs, x => x.UseAcmeTenancy())`). If you plan several provider packages, use the provider builder consistently from the start, as the NetTopologySuite plugin does.

42 A second provider: SQL Server

The package `Acme.EntityFrameworkCore.Tenancy.SqlServer` implements the same idea with SQL Server's means. The differences show what a plugin really has to solve anew per database:

Aspect	PostgreSQL	SQL Server
Tenant in the session	`set_config(name, value, false)`	`sp_set_session_context`
`CurrentTenant()` in SQL	`nullif(current_setting(...), '')::uuid`	`CAST(SESSION_CONTEXT(N'…') AS uniqueidentifier)`
Column type	`uuid`	`uniqueidentifier`
RLS mechanism	`CREATE POLICY` per table	Predicate function + `CREATE SECURITY POLICY`
Read / write	`USING` / `WITH CHECK`	`FILTER` / `BLOCK ... AFTER INSERT/UPDATE`
Table owner	Only bound with `FORCE`	No FORCE; predicates are part of the query
`DROP TABLE`	Policies disappear with it	Policy must go first (SCHEMABINDING)
DDL in batches	Transactional, freely combinable	`CREATE FUNCTION/SCHEMA` alone in a batch: `EXEC(N'...')`

Activation through the provider builder

Usage and Infrastructure/SqlServerTenancyOptionsBuilderExtensions.cs (shortened)

```
services.AddAcmeTenancy<HttpTenantProvider>();                    // core package
services.AddDbContext<ShopContext>((sp, o) => o
    .UseSqlServer(connectionString, sql => sql.UseAcmeTenancy(sp)));

public static SqlServerDbContextOptionsBuilder UseAcmeTenancy(
    this SqlServerDbContextOptionsBuilder sqlServerOptionsBuilder,
    string sessionKey = "TenantId",
    string policySchema = "acme")
{
    // From the provider builder to the actual DbContextOptionsBuilder
    var optionsBuilder =
        ((IRelationalDbContextOptionsBuilderInfrastructure)sqlServerOptionsBuilder)
        .OptionsBuilder;

    var extension = optionsBuilder.Options.FindExtension<SqlServerTenancyOptionsExtension>()
                    ?? new SqlServerTenancyOptionsExtension();
    // ... validate and set settings ...
    ((IDbContextOptionsBuilderInfrastructure)optionsBuilder)
        .AddOrUpdateExtension(extension.WithSettings(settings));

    return sqlServerOptionsBuilder;
}
```

Migrations: security policy instead of policy expression

For SQL Server, the convention only records the tenant column; the SQL is produced in the generator. The predicate function is the same for all tables and is created idempotently:

Migrations/Internal/SqlServerTenancyMigrationsSqlGenerator.cs (excerpt)

```
protected virtual void CreatePolicy(string table, string? schema, string column,
    MigrationCommandListBuilder builder)
{
    var function = Delimit(PredicateFunctionName, Settings.PolicySchema);
    var target = Delimit(table, schema);
    var predicate = $"{function}({Delimit(column)})";

    // CREATE SCHEMA/FUNCTION must be alone in a batch -> EXEC(N'...')
    Statement(builder, $"IF SCHEMA_ID({Literal(Settings.PolicySchema)}) IS NULL "
        + $"EXEC({Literal($"CREATE SCHEMA {Delimit(Settings.PolicySchema)}")})");

    var functionSql =
        $"CREATE FUNCTION {function}(@TenantId uniqueidentifier) "
        + "RETURNS TABLE WITH SCHEMABINDING AS RETURN SELECT 1 AS [Allowed] "
        + $"WHERE @TenantId = CAST(SESSION_CONTEXT({Literal(Settings.SessionKey)}) "
        + "AS uniqueidentifier)";
    Statement(builder, $"IF OBJECT_ID({Literal(function)}, N'IF') IS NULL "
        + $"EXEC({Literal(functionSql)})");

    Statement(builder,
        $"CREATE SECURITY POLICY {PolicyName(table)} "
        + $"ADD FILTER PREDICATE {predicate} ON {target}, "
        + $"ADD BLOCK PREDICATE {predicate} ON {target} AFTER INSERT, "
        + $"ADD BLOCK PREDICATE {predicate} ON {target} AFTER UPDATE "
        + "WITH (STATE = ON)");
}

protected override void Generate(DropTableOperation operation, IModel? model,
    MigrationCommandListBuilder builder, bool terminate = true)
{
    DropPolicy(operation.Name, builder);       // SCHEMABINDING: the policy first
    base.Generate(operation, model, builder, terminate);
}
```

`SCHEMABINDING` only accepts two-part names. A table without an explicit schema is therefore qualified by the callers with the model's default schema or, failing that, with `dbo`; otherwise SQL Server rejects the security policy.

> **Nested SQL**
>
> The `EXEC(N'...')` contains SQL in a string literal that in turn contains literals. The nesting is produced exclusively through the string mapping's `GenerateSqlLiteral`, which doubles quotes correctly; identifiers go through `DelimitIdentifier`. Session key and schema are additionally validated during configuration.

Run time: same flow, different command

Runtime/SqlServerTenantSessionInterceptor.cs

```
public class SqlServerTenantSessionInterceptor : TenantSessionInterceptor
{
    public SqlServerTenantSessionInterceptor(ITenantProvider tenants)
        : base(tenants)
    {
    }

    protected override DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant)
    {
        var key = context?.GetService<ISqlServerTenancySingletonOptions>()
                      .Settings.SessionKey
                  ?? SqlServerTenancySettings.Default.SessionKey;

        var command = connection.CreateCommand();
        command.CommandText = "EXEC sp_set_session_context @key = @key, @value = @value";
        AddParameter(command, "@key", key);
        AddParameter(command, "@value", tenant?.Value);    // NULL: no rows visible
        return command;
    }
}
```

Deliberately not ported

The SQL Server package is a complete but leaner provider. Fluent API policies, the optional query filter, custom SQL nodes, aggregates, the snapshot code generator and scaffolding are missing there. That is a product decision, not a technical obstacle: each of these chapters can be transferred to SQL Server with the same patterns. The list belongs in the package's documentation so users know what to expect.

tests/...SqlServer.Tests/SqlServerTests.cs (excerpt)

```
[Fact]
public async Task Block_predicate_rejects_rows_of_other_tenant()
{
    await using var db = ContextFor(TenantId.New());

    var ex = await Assert.ThrowsAsync<SqlException>(() => db.Database.ExecuteSqlAsync(
        $"INSERT INTO [Orders] ([TenantId], [Total]) VALUES ({Guid.NewGuid()}, 1)"));

    Assert.Equal(33504, ex.Number);      // violation of a BLOCK PREDICATE
}
```

As with PostgreSQL, the tests run on two levels: without a database for model, query and migration SQL, and with Testcontainers against a real SQL Server. There, the application connects with its own login instead of `sa` to reproduce the production situation.

Part X

Appendices

For reference: extension points with lifetimes, files of the sample, terms and a debugging toolbox.

› Appendix A: Extension points and lifetimes

› Appendix B: Files of the sample plugin

› Appendix C: Glossary

› Appendix D: Debugging toolbox

› License

A Extension points and lifetimes

Lifetimes and cardinality according to EF Core 10's service tables (`EntityFrameworkServicesBuilder`, `EntityFrameworkRelationalServicesBuilder`). "Multiple" means: plugin interface, all registrations are used.

Service	Lifetime	Kind	Ch.
`IConventionSetPlugin`	Scoped	multiple	8
`ISingletonOptions`	Singleton	multiple	5
`ITypeMappingSourcePlugin`	Singleton	multiple	10
`IRelationalTypeMappingSourcePlugin`	Singleton	multiple	10
`IMethodCallTranslatorPlugin`	Scoped	multiple	12
`IMemberTranslatorPlugin`	Scoped	multiple	12
`IAggregateMethodCallTranslatorPlugin`	Scoped	multiple	38
`IEvaluatableExpressionFilterPlugin`	Singleton	multiple	13
`IInterceptor` (injected)	Scoped	multiple	15
`ISingletonInterceptor`	Singleton	multiple	15
`IRelationalAnnotationProvider`	Singleton	single	18
`IMigrationsSqlGenerator`	Scoped	single	18
`IMigrationsModelDiffer`	Scoped	single	19
`IModelValidator`	Singleton	single	9
`IModelCustomizer`	Singleton	single	–
`IModelCacheKeyFactory`	Singleton	single	31
`ISqlGenerationHelper`	Singleton	single	–
`IRelationalTypeMappingSource`	Singleton	single	10
`ISqlExpressionFactory`	Scoped	single	12
`IQuerySqlGeneratorFactory`	Singleton	single	14, 37
`IModificationCommandFactory`	Singleton	single	29
`ICommandBatchPreparer`	Scoped	single	29
`IModificationCommandBatchFactory`	Scoped	single	29
`IUpdateSqlGenerator`	Singleton	single	29
`IRelationalParameterBasedSqlProcessorFactory`	Scoped	single	37

Design-time services (via IDesignTimeServices)

Service	Purpose
`IAnnotationCodeGenerator`	Annotations as Fluent API in snapshot and scaffolding
`ICSharpRuntimeAnnotationCodeGenerator`	Filter annotations in the compiled model or translate complex values (chapter 39)
`ICSharpMigrationOperationGenerator`	Custom operations as C# in migrations
`IDatabaseModelFactory`	Read the schema for scaffolding (chapter 40)
`IProviderConfigurationCodeGenerator`	UseXyz call in the scaffolded context

B Files of the sample plugin

File	Content	Ch.
Core package		
`TenantId.cs`, `TenantIdConverter.cs`	Strongly typed ID, converter	10
`TenantIdJsonReaderWriter.cs`	JSON support	10
`ITenantOwned.cs`, `ITenantProvider.cs`	Marker, tenant source	3
`NoTenantIsolationAttribute.cs`	Data annotation	8
`TenancyDbFunctionsExtensions.cs`	`EF.Functions.CurrentTenant()`	12
`Diagnostics/*`	Event IDs, definitions, logger extensions	16
`Interceptors/TenantSaveChanges Interceptor.cs`	SaveChanges logic	15
`Interceptors/TenantSessionInterceptor.cs`	Abstract base for session commands	15, 41
`TenancyDatabaseFacadeExtensions.cs`	`Database.ApplyTenantSession()`	15
`Metadata/*` (core)	Runtime annotation, IsTenantProperty, candidate lookup	39
`Query/TenancyEvaluatableExpressionFilter Plugin.cs`	Filter for all packages	13, 41
`DependencyInjection/*`	AddAcmeTenancy	6, 41
`ValueGeneration/TenantIdValueGenerator.cs`, `…Factory.cs`	Value generation	28
`Conventions/TenantIdKeyValueGeneration Convention.cs`	Generator for keys	28
PostgreSQL package		
`Infrastructure/TenancyOptionsExtension.cs`	Extension, info, ApplyServices	4, 5
`Infrastructure/TenancySettings.cs`	Settings as a record	4
`Infrastructure/ITenancySingletonOptions.cs`	Singleton options	5
`Infrastructure/TenancyOptionsBuilder.cs`	Options builder	6
`Infrastructure/Tenancy*Extensions.cs`	UseAcmeTenancy	6
`Infrastructure/TenancyIdentifiers.cs`	Identifier validation	8, 19
`Storage/*`	Type mapping and plugin	10
`Query/*`	Translators, functions	12, 13
`Metadata/*`	Annotations, RlsPolicy, PolicyCommand, annotation provider	7, 18
`Builders/*`	Fluent API, PolicyBuilder	7
`Conventions/*`	Isolation, attribute, validation, plugin	8, 9
`Migrations/*`	SQL generator, custom operations	18, 19
`Design/*`, `Properties/AssemblyInfo.cs`	Code generator, design time	20
`Runtime/TenantConnectionInterceptor.cs`	Set the session setting	15

File	Content	Ch.
`Query/Expressions/*`, `Query/DistinctFromTranslator.cs`	Custom SQL node	37
`Query/Internal/*`	SQL generator, nullability processor, factories	37
`Query/EveryAggregateTranslator.cs`	Aggregate translator	38
`Design/Internal/TenancyCSharpRuntime…Generator.cs`	Compiled model filter	39
`Scaffolding/*`	Read RLS from the database	40
SQL Server package		
`Infrastructure/*`	Extension, provider-builder activation	42
`Storage/*`, `Query/*`	uniqueidentifier, SESSION_CONTEXT	42
`Conventions/*`, `Metadata/*`	Isolation, column annotation	42
`Migrations/*`	Predicate function, security policy	42
`Runtime/*`	sp_set_session_context	42
Further		
`samples/ShopSample`	Console sample with design-time factory	3, 32
`samples/ShopApi`	ASP.NET Core sample (minimal API)	35
`.github/workflows/ci.yml`	Build, tests, handbook	36
`PublicAPI.*.txt`	Public API analyzer	36
`docs/handbook/*`	Sources of this book (German and English)	–
`tests/...Tests`	Unit tests without a database	23
`tests/.../MinimalPluginTests.cs`	The mini plugin	2
`tests/.../CompositionTests.cs`	Decorator example (audit plugin)	5
`tests/.../DebuggingTests.cs`	Debugging techniques	App. D
`tests/.../SchemaPerTenantTests.cs`	Model cache key per schema	31
`benchmarks/...`	BenchmarkDotNet project for the SaveChanges logic	34
`tests/...IntegrationTests`	Testcontainers tests against PostgreSQL	15, 23, 27, 33, 40
`tests/...SqlServer.Tests`	Unit and Testcontainers tests for SQL Server	42

C Glossary

Term	Explanation
Annotation	Key-value metadata on a model element.
ConfigurationSource	Origin of a piece of metadata (Explicit, DataAnnotation, Convention); determines who may override whom.
Convention	Rule that adapts the model automatically while it is built.
Decorator	Service that wraps the previously registered implementation and delegates to it; composes with other plugins (chapter 5).
Design-time model	Complete model with all annotations; basis for migrations. Counterpart: the lean runtime model.
Differ	`IMigrationsModelDiffer`; compares two relational models.
Funcletizer	First query stage; evaluates expressions without data reference in .NET.
Nullability processor	Query stage that evaluates NULL semantics and optimizes comparisons; must know custom SQL nodes.
SqlExpression	Node in the SQL expression tree; custom nodes for syntax that is not a function.
Runtime annotation	Annotation read at run time, which must therefore survive in the runtime and compiled model.
Internal service provider	EF's own DI container per configuration.
Pubternal	Publicly visible but internal API without compatibility guarantees (EF1001).
Relational model	Tables, columns, indexes, derived from the EF model.
RLS	Row Level Security: row filters enforced by the database itself.
Snapshot	Generated C# code with the model state of the last migration.
Type mapping	Connection between CLR type and database type including conversion, comparison and literals.

D Debugging toolbox

When a plugin does not behave as expected, the question is almost always one of five. Each technique below is checked in `DebuggingTests`.

Question	Technique
Is my annotation in the model?	`ToDebugString(MetadataDebugStringOptions.LongDefault)` on the design-time model
Which implementation of a single service won?	`context.GetService<IMigrationsSqlGenerator>().GetType()`
Is my convention registered?	`GetService<IConventionSetBuilder>().CreateConventionSet()`, then e.g. `ModelFinalizingConventions`
What does the query look like before and after translation?	`query.Expression.Print()` and `query.ToQueryString()`
Does my plugin log?	`LogTo(lines.Add, [eventId], LogLevel.Debug)`, restricted to the plugin's event IDs

Debugging snippets (checked in tests/.../DebuggingTests.cs)

```
using var ctx = CreateContext();

// 1. Model: annotations, results of conventions, type mappings
var model = ctx.GetService<IDesignTimeModel>().Model;
Console.WriteLine(model.ToDebugString(MetadataDebugStringOptions.LongDefault));

// 2. Services: which replacement won?
Console.WriteLine(ctx.GetService<IMigrationsSqlGenerator>().GetType());

// 3. Conventions: is mine in the list?
var conventions = ctx.GetService<IConventionSetBuilder>().CreateConventionSet();
Console.WriteLine(string.Join("\n", conventions.ModelFinalizingConventions));

// 4. Queries: LINQ tree before, SQL after translation
var query = ctx.Orders.Where(o => o.TenantId == EF.Functions.CurrentTenant());
Console.WriteLine(query.Expression.Print());
Console.WriteLine(query.ToQueryString());
```

More tools

Tool	Use
Debugger with Source Link, "Just My Code" off	Step into EF Core and Npgsql; both ship symbols and Source Link. A breakpoint in your own convention shows when and how often it runs.
`DebugView` in the debugger	Model, entity types, query objects and the change tracker show the same texts as `ToDebugString`.
`dotnet ef ... --verbose`	Shows which assemblies and design-time services the tools load; the first stop when a design-time service seems to be ignored.
`EnableSensitiveDataLogging`, `EnableDetailedErrors`	Parameter values and detailed errors in the log; for development only.

Tool	Use
`ManyServiceProvidersCreatedWarning`	EF warns when too many internal providers are created: almost always a hash mistake (chapter 4).
EF Core's source code on GitHub	The authoritative documentation for internal APIs; the folder structure matches the namespaces.

> **Reproduce in a test, not in the app**
>
> Almost every plugin problem can be reproduced without a database: build a context with the plugin and look at the model, the resolved services or `ToQueryString()`. Such a test iterates faster than the application and remains as a regression test afterwards.

License

This handbook and the sample code in the repository are licensed under the MIT License. You may use, copy, modify and distribute them, also commercially, provided that the following copyright and license notice is included.

Copyright (c) 2026 guenni81

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

The DejaVu fonts embedded in this PDF are under their own free license (Bitstream Vera Fonts license; the DejaVu changes are in the public domain); see `docs/handbook/fonts/LICENSE-DejaVu.txt` in the repository.

Product and company names mentioned in this book are trademarks of their respective owners, for example Entity Framework, .NET and SQL Server (Microsoft) and PostgreSQL (PostgreSQL Community Association). They are used for descriptive purposes only. This handbook is an independent project and is not affiliated with or endorsed by Microsoft, the PostgreSQL Global Development Group or the Npgsql project.